

Universidad de Castilla-La Mancha

Escuela Superior de Ingeniería Informática de Albacete

Escuela Superior de Informática de Ciudad Real

Trabajo Fin de Máster

Máster Universitario en Inteligencia Artificial

Implementación de un sistema de detección de amenazas aéreas basada en IA, dispositivo móvil y cámara deportiva

Óscar Domínguez Ocaña

Octubre, 2026

Escuela Superior de Ingeniería Informática Albacete

Escuela Superior de Informática

TRABAJO FIN DE MÁSTER

Máster Universitario en Inteligencia Artificial

Implementación de un sistema de detección de amenazas aéreas basada en IA, dispositivo móvil y cámara deportiva

Autor: Óscar Domínguez Ocaña

Director: José Pascual Molina Massó

Codirector: Arturo Simón García Jiménez

UCLM Universidad Castilla-La Mancha

Octubre, 2026

A mi familia

Declaración de autoría

Yo, Óscar Domínguez Ocaña, con DNI 05711111G, declaro que soy el único autor del trabajo fin de máster titulado "Implementación de un sistema de detección de amenazas aéreas basada en IA, dispositivo móvil y cámara deportiva", que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores.

Ciudad Real, a 7 de octubre de 2026

Fdo.: Óscar Domínguez Ocaña

Resumen

Las colisiones en el aire entre aeronaves (*midair collisions*), aunque poco frecuentes, siguen siendo uno de los sucesos más graves de la aviación por sus consecuencias prácticamente irreversibles, especialmente entre aeronaves ligeras no equipadas con sistemas de comunicación cooperativos. En estos escenarios el piloto depende de la observación visual directa (*see-and-avoid*), un método limitado por la fatiga y por la atención simultánea a otras tareas de vuelo.

Este Trabajo Fin de Máster desarrolla un sistema de detección de amenazas aéreas en tiempo real, implementado como aplicación Android que se comunica con una cámara externa GoPro Hero 12 mediante Bluetooth Low Energy, WiFi y HTTP a través de la interfaz *Open GoPro*. El sistema recibe el flujo de vídeo H.264, ejecuta la inferencia *on-device* de un modelo YOLOv8n entrenado con el conjunto *Airborne Object Tracking* de Amazon Prime Air, y superpone las detecciones sobre el vídeo mediante una interfaz orientada al contexto de aviación.

El modelo alcanza sobre el conjunto de validación una *mAP@0.5* de 0.993 en la clase *aircraft*, con latencia de inferencia entre 60 y 130 ms en dispositivo móvil. Durante el proyecto se han evaluado y descartado alternativas técnicas como Google Media3, una implementación propia sobre `MediaCodec` y la división por secuencia del *dataset*, documentando las razones de la solución final.

Palabras clave: detección de objetos, visión por computador, YOLOv8, LiteRT, Android, GoPro, seguridad aérea.

Abstract

Midair collisions between aircraft, though infrequent, remain one of the most serious events in aviation due to their almost irreversible consequences, particularly among light aircraft that are not equipped with cooperative communication systems. In these scenarios the pilot relies on direct visual observation (*see-and-avoid*), a method constrained by fatigue and by the need to attend simultaneously to other flight tasks.

This Master's Thesis develops a real-time detection system for airborne threats, implemented as an Android application that communicates with an external GoPro Hero 12 camera via Bluetooth Low Energy, WiFi and HTTP through the *Open GoPro* interface. The system receives the H.264 video stream, performs *on-device* inference of a YOLOv8n model trained with Amazon Prime Air's *Airborne Object Tracking* dataset, and overlays the detections on the video through a user interface oriented to the aviation context.

The model achieves on the validation set an *mAP@0.5* of 0.993 on the *aircraft* class, with inference latency between 60 and 130 ms on the mobile device. During the project, technical alternatives such as Google Media3, a custom implementation over `MediaCodec` and per-sequence *dataset* splitting have been evaluated and discarded, documenting the reasons for the final solution.

Keywords: object detection, computer vision, YOLOv8, LiteRT, Android, GoPro, aviation safety.

Agradecimientos

A José Pascual y Arturo Simón, por su orientación y disponibilidad durante el desarrollo del trabajo.

A la Escuela Superior de Informática de la UCLM, por haber facilitado el marco académico para la realización de este trabajo.

A mi familia, por su apoyo incondicional durante todo el máster y por haberme permitido dedicar el tiempo necesario a este trabajo, compaginándolo con la actividad profesional.

A Luis, por su paciencia con las horas que este proyecto se ha llevado.

A José Manuel "Chema", con quien he compartido este máster y las dificultades añadidas que han acompañado al curso; haberlo recorrido en buena compañía ha hecho que merezca la pena.

A Flox, por su gesto generoso y desinteresado justo cuando el proyecto más lo necesitaba.

Y, en particular, a quien ha compartido conmigo cada una de esas horas, por su paciencia y por estar siempre.

Índice general

Acrónimos . . . . . **xxiii**

1 Introducción **1**

1.1 Motivación y contexto 1

1.1.1 Propuesta del TFM *2*

1.2 Estructura del documento 3

2 Objetivos **5**

2.1 Objetivo general 5

2.2 Objetivos específicos 5

3 Estado del arte **7**

3.1 Prevención de colisiones en el aire 7

3.2 Detección de aeronaves mediante visión por computador 8

3.3 Conjuntos de datos 8

3.4 Inferencia en el dispositivo 9

3.5 Soluciones existentes y posicionamiento 9

3.5.1 Aerial Guard *9*

3.5.2 Iris Automation Casia *10*

3.5.3 AirTrack *11*

3.5.4 Posicionamiento de la propuesta *11*

4 Metodología de trabajo **13**

4.1 Enfoque metodológico 13

4.2 Planificación temporal 14

4.3 Marco tecnológico 15

4.3.1 Aplicación móvil *15*

4.3.2 Entrenamiento del modelo . . . 16
4.3.3 Control de versiones y colaboración . . . 16
4.4 Recursos utilizados . . . 16
4.4.1 Recursos hardware. . . . 16
4.4.2 Recursos software y servicios en la nube. . . . 17
4.5 Limitaciones . . . 17
5 Detección de amenazas aéreas mediante visión por computador . . 19
5.1 Comunicación con la cámara . . . 19
5.1.1 Emparejamiento y control por Bluetooth Low Energy . . . 20
5.1.2 Activación del punto de acceso y conexión WiFi . . . 21
5.1.3 Control por HTTP mediante la interfaz Open GoPro . . . 22
5.2 Pipeline de *streaming* de vídeo . . . 22
5.2.1 Evaluación de alternativas descartadas . . . 23
5.2.2 Adopción de LibVLC . . . 24
5.2.3 Captura de fotogramas para inferencia . . . 24
5.3 Modelo de detección . . . 26
5.3.1 Conjunto de datos AOT: descarga y análisis . . . 26
5.3.2 Preparación del subconjunto y formato YOLO . . . 28
5.3.3 División train/val: primera iteración y corrección . . . 29
5.3.4 Entrenamiento del modelo . . . 31
5.3.5 Métricas finales y decisiones sobre el modelo . . . 32
5.3.6 Exportación al formato LiteRT . . . 33
5.4 Integración modelo-aplicación . . . 34
5.4.1 Adaptación del pipeline *de inferencia* . . . 34
5.4.2 Postprocesado: decodificación y Non-Maximum Suppression . . . 35
5.4.3 Muestreo periódico y control de backpressure . . . 35
5.5 Interfaz de usuario . . . 36
5.5.1 Diseño visual: tema "Aviación" . . . 36
5.5.2 Flujo progresivo de conexión . . . 37
5.5.3 Superposición de detecciones sobre el vídeo. . . . 38
5.5.4 Seguimiento simple entre fotogramas . . . 40
6 Resultados . . . 41
6.1 Rendimiento del modelo de detección . . . 41
6.2 Caracterización en el dispositivo móvil . . . 42

6.3 Validación empírica con GoPro real . . . 43
6.3.1 Protocolo 1: proyección de imágenes del propio dataset . . . *43*
6.3.2 Protocolo 2: escenarios reales al aire libre . . . *44*

7 Conclusiones y trabajo futuro . . . 47
7.1 Cumplimiento de los objetivos . . . 47
7.2 Conclusiones generales . . . 48
7.3 Competencias adquiridas . . . 49
7.4 Líneas de trabajo futuro . . . 50
7.4.1 Ampliación del conjunto de entrenamiento . . . *50*
7.4.2 Distinción de escenarios airprox . . . *50*
7.4.3 Modo operativo minimalista . . . *50*
7.4.4 Certificación y despliegue operativo . . . *50*

Referencia bibliográfica . . . 52

A Código fuente relevante . . . 55
A.1 Preprocesado CHW y ejecución del modelo . . . 55
A.2 Muestreo periódico de fotogramas . . . 55
A.3 Conversión del *dataset* AOT al formato YOLO . . . 57
A.4 Configuración de LibVLC y adjunción de vista . . . 58

B Guía de instalación y despliegue . . . 59
B.1 Repositorio del proyecto . . . 59
B.2 Requisitos previos . . . 60
B.3 Compilación e instalación de la aplicación . . . 60
B.4 Configuración inicial de la cámara . . . 61
B.5 Uso de la aplicación . . . 61
B.6 Reentrenamiento del modelo . . . 62
B.6.1 Descarga y preparación del dataset . . . *62*
B.6.2 Entrenamiento en Google Colab . . . *62*
B.6.3 Integración del nuevo modelo en la aplicación . . . *63*
B.7 Consideraciones de compatibilidad hardware . . . 63

C Estimación de costes . . . 65
C.1 Coste de despliegue del sistema . . . 65
C.2 Comparación con Aerial Guard . . . 66

C.3 Coste de reentrenamiento del modelo 66
C.4 Conclusiones del análisis de costes 67

Índice de figuras

1.1 Escenarios de aproximación entre aeronaves 2
1.2 Arquitectura general del sistema propuesto 3

3.1 Interfaz de Aerial Guard 10
3.2 Sistema Iris Automation Casia 10
3.3 Sistema AirTrack 11

4.1 Cronograma de desarrollo del TFM 15
4.2 Marco tecnológico del proyecto 17

5.1 Diagrama de secuencia de la comunicación con la GoPro 20
5.2 Pipeline de *streaming* y captura de fotogramas 25
5.3 Estructura de anotaciones del *dataset* AOT 27
5.4 Fotogramas representativos de las secuencias empleadas 27
5.5 Distribución de anotaciones por clase en el subconjunto AOT 28
5.6 Conversión de anotaciones AOT al formato YOLOv8 29
5.7 Análisis de las anotaciones tras la conversión al formato YOLO 30
5.8 Estrategias de división train/val 31
5.9 Convergencia del entrenamiento de YOLOv8n 32
5.10 Pipeline de inferencia en la aplicación Android 36
5.11 Aspecto general de la aplicación con tema “Aviación” 37
5.12 Revelación progresiva de secciones en `DeviceCard` 39
5.13 Superposición del overlay de detecciones sobre el vídeo 40
5.14 Historial de detecciones recientes 40

6.1 HUD con latencia de inferencia 43
6.2 Detección de aeronave comercial en prueba de campo 45

B.1 Código QR de acceso al repositorio del proyecto 59

Índice de tablas

5.1 Métricas de validación del modelo final 33

6.1 Cumplimiento de umbrales mínimos para el modelo 41
6.2 Latencia de inferencia por dispositivo 42
6.3 Resultados del protocolo 1: proyección de imágenes del AOT 44

C.1 Coste estimado de despliegue del sistema 65
C.2 Comparación de coste y características con Aerial Guard 66
C.3 Estimación de coste de reentrenamiento según escenario 67

Índice de listados de código

A.1 GoProDetector: preprocesado de `Bitmap` a tensor CHW 56
A.2 GoProDetector: implementación manual de NMS 56
A.3 FrameSampler: captura de fotograma sobre `TextureView` 57
A.4 Conversión de anotaciones AOT al formato YOLO 57
A.5 GoProStreamPlayer: configuración de LibVLC 58

Acrónimos

ACAS	Airborne Collision Avoidance System
ADS-B	Automatic Dependent Surveillance – Broadcast
AGP	Android Gradle Plugin
AOT	Airborne Object Tracking
AP	Access Point
API	Application Programming Interface
BLE	Bluetooth Low Energy
BVLOS	Beyond Visual Line of Sight
CCCD	Client Characteristic Configuration Descriptor
CHW	Channels, Height, Width
CPU	Central Processing Unit
DAA	Detect and Avoid
EASA	European Union Aviation Safety Agency
FAA	Federal Aviation Administration
FLARM	Flight Alarm
GATT	Generic Attribute Profile
GPU	Graphics Processing Unit
HTTP	HyperText Transfer Protocol
HUD	Head-Up Display
HWC	Height, Width, Channels

IA	Inteligencia Artificial
IoU	Intersection over Union
JDK	Java Development Kit
LiteRT	Lite Runtime (anteriormente TensorFlow Lite)
mAP	mean Average Precision
MPEG-TS	MPEG Transport Stream
NMS	Non-Maximum Suppression
SDK	Software Development Kit
SoC	System-on-Chip
SSID	Service Set Identifier
TCAS	Traffic Alert and Collision Avoidance System
TFM	Trabajo Fin de Máster
TLS	Transport Layer Security
UDP	User Datagram Protocol
UUID	Universally Unique Identifier
YOLO	You Only Look Once

1. Introducción

Las colisiones en el aire entre aeronaves (*midair collisions*), aunque son poco frecuentes, siguen siendo uno de los sucesos más graves de la aviación por sus terribles consecuencias, y han ganado relevancia con la creciente presencia de aeronaves no tripuladas en el espacio aéreo. En este contexto, la seguridad aérea ha experimentado un importante progreso en los últimos años, tanto a nivel de *hardware* con la aparición de nuevos dispositivos de detección de bajo coste, como a nivel de *software* con el desarrollo de algoritmos de visión por computador y aprendizaje automático que facilitan tareas que antes requerían exclusivamente de la percepción humana.

Esta evolución ha dado lugar a un interés por aplicar estas tecnologías a la detección de tráfico aéreo, especialmente en los casos en los que las aeronaves implicadas no disponen de sistemas de comunicación cooperativos entre sí (transpondedores, ADS-B[1], FLARM [1], etc.), lo que obliga a depender del método tradicional *see-and-avoid*[2], es decir, de la observación visual directa por parte del piloto. La motivación de este TFM es precisamente ayudar a los pilotos a detectar visualmente las amenazas aéreas potenciales, evitando así posibles colisiones en el aire.

1.1. Motivación y contexto

Este tipo de sucesos es una preocupación constante para agencias como la europea EASA [2] y para organismos nacionales como la Agencia Estatal de Seguridad Aérea [3],

[1] *Automatic Dependent Surveillance–Broadcast*: sistema de vigilancia cooperativa mediante el cual la aeronave difunde periódicamente su posición, calculada por sistemas de navegación por satélite, para que otras aeronaves y las estaciones en tierra puedan recibirla.

[2] Principio operativo por el que el piloto es responsable de detectar visualmente el tráfico circundante y maniobrar para evitarlo.

preocupación que ha aumentado con la creciente introducción de aeronaves no tripuladas en nuestros cielos.

En particular, existen unos escenarios de aproximación entre aeronaves que, al igual que ocurre en seguridad vial, dan lugar a reglas de paso (*right-of-way*) para evitar colisiones: aproximación por alcance (*close-in*), aproximación frontal (*head-to-head*) y aproximación por cruce (*crossing*). Ya existen soluciones que abordan la detección visual en estos escenarios, como Aerial Guard [4, 5], si bien se trata de sistemas orientados principalmente a operadores profesionales, con la particularidad de que emplean la cámara de un dispositivo móvil en lugar de una cámara externa dedicada. La Figura 1.1 ilustra los tres escenarios típicos de aproximación entre aeronaves.

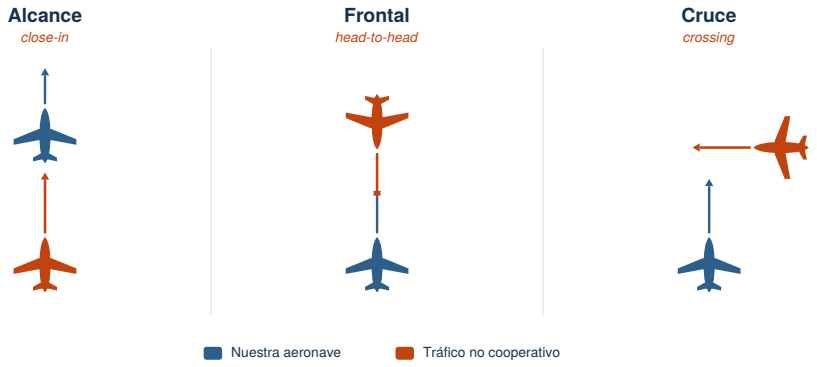


Figura 1.1: Escenarios de aproximación entre aeronaves: close-in, head-to-head y crossing

1.1.1. Propuesta del TFM

El propósito de este trabajo es el desarrollo de una aplicación Android, implementada con Android Studio, que permita detectar posibles amenazas aéreas a partir del vídeo capturado por una cámara externa de tipo GoPro, comunicándose con ella a través de su interfaz Open GoPro [6, 7]. Esta es una diferencia relevante respecto a soluciones como Aerial Guard, ya que el uso de una cámara dedicada permite una mayor calidad de imagen, mayor flexibilidad de posicionamiento en la aeronave y no compromete el uso del propio dispositivo móvil para otras funciones durante el vuelo.

La detección de amenazas se llevará a cabo mediante un modelo de aprendizaje auto-

mático ejecutado con LiteRT de Google [8][3], entrenado sobre el conjunto de datos *Airborne Object Tracking* (AOT), publicado por Amazon Prime Air a través de la plataforma AIcrowd [9] en el marco del *Airborne Object Tracking Challenge*. Se trata de casi cinco mil secuencias de vuelo reales grabadas con cámaras frontales, un conjunto lo bastante grande y variado como para entrenar y validar el modelo sin tener que crear uno propio desde cero. En la Figura 1.2 se muestra la arquitectura general del sistema y sus componentes principales.

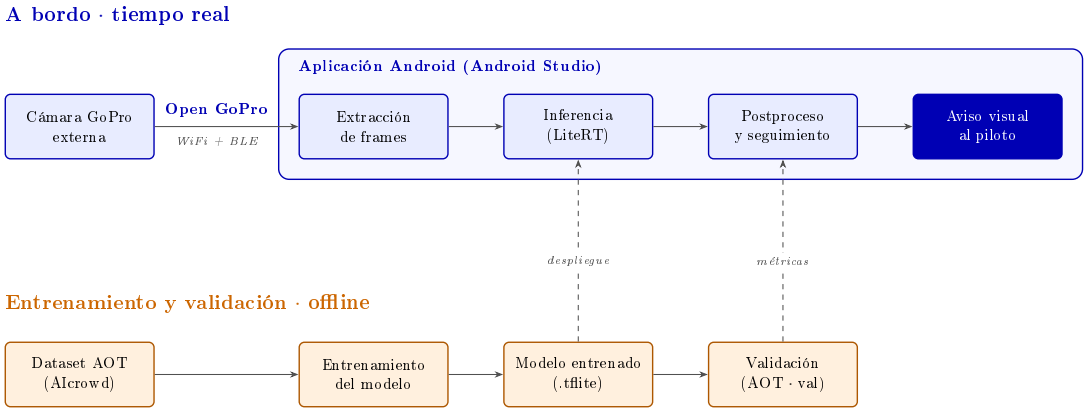


Figura 1.2: Arquitectura general del sistema de detección propuesto en el presente TFM

La idea es ofrecer una herramienta que ayude al piloto a decidir, sin reemplazar la observación visual tradicional. Le da un extra de conciencia situacional y reduce el riesgo de colisión en vuelo gracias a una especie de "ojo artificial" que lo complementa. Además funciona sin depender de que las demás aeronaves cooperen.

1.2. Estructura del documento

El presente documento se estructura de la siguiente forma:

- **Capítulo 1: Introducción**. Se introduce el contexto en el que se enmarca el presente TFM, las principales problemáticas y necesidades, así como una visión global de la propuesta.
- **Capítulo 2: Objetivos**. Descripción del objetivo general de este TFM, así como los objetivos parciales que conducen a su consecución.

[3]Motor de Google para ejecutar modelos de aprendizaje automático en dispositivos con recursos limitados, como teléfonos móviles; anteriormente conocido como TensorFlow Lite.

- **Capítulo 3: Estado del arte**. Se exponen los antecedentes y se realiza un estudio de las soluciones existentes relacionadas con la propuesta.
- **Capítulo 4: Metodología de trabajo**. Se presenta el método de trabajo seguido, el marco tecnológico empleado, los medios utilizados y las limitaciones del proyecto.
- **Capítulo 5: Detección de amenazas aéreas mediante visión por computador**. Decisiones de diseño y desarrollo del sistema, incluyendo el entrenamiento del modelo y la implementación de la aplicación Android.
- **Capítulo 6: Resultados**. Rendimiento del modelo de detección y funcionamiento de la aplicación en los escenarios evaluados.
- **Capítulo 7: Conclusiones y trabajo futuro**. Conclusiones extraídas tras la elaboración del proyecto, posibles mejoras y futuras líneas de trabajo.

Tras los capítulos se presenta la bibliografía consultada para realizar este Trabajo Fin de Máster.

2. Objetivos

En este capítulo se describe el objetivo general planteado en el presente Trabajo Fin de Máster (TFM), y los diferentes objetivos específicos planteados para la consecución del objetivo general.

2.1. Objetivo general

Partiendo del contexto descrito en el capítulo anterior, donde las aeronaves no siempre cuentan con sistemas cooperativos y el piloto depende de su propia vista para identificar posibles objetos con los que pueda colisionar, el objetivo general de este trabajo es diseñar e implementar una alternativa de "ojo artificial" que no dependa de que las demás aeronaves cooperen. Esta alternativa se materializa en una aplicación Android que, a partir de las imágenes captadas por una cámara externa GoPro y usando la biblioteca de inteligencia artificial LiteRT, identifica tráficos en el aire, sirviendo así de apoyo al piloto y al resto de tecnologías cooperativas.

2.2. Objetivos específicos

Además del objetivo general, se han definido una serie de objetivos específicos que, conjuntamente, permitirán alcanzarlo.

1. **Comunicación entre el dispositivo móvil y la cámara deportiva externa**. Emparejar por Bluetooth de baja energía (BLE) con la GoPro, activar por código su punto de acceso WiFi, obtener en tiempo real el vídeo que capta la cámara y controlar a distancia sus funciones principales a través de la interfaz Open GoPro.

2. **Creación y entrenamiento de un modelo de aprendizaje automático** para la detección de aeronaves, optimizado para su ejecución con LiteRT sobre dispositivos móviles, buscando un equilibrio entre precisión y rendimiento en tiempo real que sobre todo minimice los falsos negativos. Como base de entrenamiento y validación se utiliza el conjunto de datos *Airborne Object Tracking* (AOT), publicado por Amazon Prime Air en la plataforma AIcrowd.
3. **Integración del modelo en la aplicación Android**, de forma que el procesamiento del vídeo procedente de la cámara externa se realice en tiempo real y con una latencia lo suficientemente baja como para resultar útil al piloto.
4. **Diseño de una interfaz de usuario** que muestre las detecciones destacando la posición de las aeronaves detectadas y proporcionando información complementaria que apoye la toma de decisiones del piloto, incluyendo su posición relativa aproximada dentro del campo de visión capturado.
5. **Experimentación y validación del sistema** mediante métricas objetivas sobre un conjunto de imágenes independiente al de entrenamiento, así como pruebas empíricas sobre el sistema completo integrado con la cámara real, midiendo tanto el rendimiento del modelo (precisión, sensibilidad) como el de la aplicación (velocidad de procesamiento, latencia), con el fin de valorar la viabilidad real del sistema.

3. Estado del arte

En este capítulo se revisan los antecedentes y las soluciones existentes relacionadas con la detección de tráfico aéreo no cooperativo: los sistemas actuales de prevención de colisiones, las técnicas de visión por computador aplicadas al problema, los conjuntos de datos disponibles y las soluciones comerciales y académicas más relevantes. Finalmente se sitúa la propuesta de este TFM respecto a ellas.

3.1. Prevención de colisiones en el aire

Como se introdujo en el Capítulo 1, la prevención de colisiones descansa sobre el principio *see-and-avoid*, por el que el piloto debe detectar visualmente el tráfico y maniobrar según las reglas de paso (*right-of-way*). Los sistemas de apoyo actualmente en uso se dividen en dos familias según requieran o no la colaboración de la aeronave detectada.

Los **sistemas cooperativos** exigen que ambas aeronaves porten equipamiento compatible que intercambia información: el transpondedor y los sistemas TCAS/ACAS asociados, la vigilancia dependiente automática de radiodifusión (ADS-B), o, en aviación ligera, FLARM [1]. La limitación principal es que muchas aeronaves como los planeadores, ultraligeros o aeronaves no tripuladas pequeñas, carecen de ellos o pueden sufrir fallos, en cuyo caso la seguridad recae de nuevo sobre la observación del piloto [2].

Ante esta limitación y la creciente integración de aeronaves no tripuladas, se ha impulsado la automatización del *see-and-avoid* en lo que se conoce como *detect and avoid* (DAA): sistemas que detectan automáticamente el tráfico y alertan o maniobran. Este enfoque, muy ligado a las operaciones más allá de la línea visual (BVLOS) y a estándares como ASTM F3442 [10], es también aplicable como apoyo a la decisión del piloto de

aeronaves tripuladas, línea en la que se enmarca este trabajo.

3.2. Detección de aeronaves mediante visión por computador

La detección no cooperativa puede abordarse con radar, sensores acústicos, LiDAR o cámaras. Las cámaras electroópticas se han consolidado como opción preferente por su tamaño, peso, consumo y coste, y por tratarse de un sensor pasivo [10], lo que motiva el uso en este TFM de una cámara externa GoPro.

Los primeros trabajos empleaban técnicas clásicas como el filtrado morfológico-temporal, sustracción de fondo o flujo óptico [11], eficaces con la aeronave recortada contra el cielo pero con dificultades sobre fondos con *clutter* de terreno (*below horizon*) y con aeronaves sin movimiento relativo aparente [12]. La aparición de los detectores basados en redes neuronales convolucionales desplazó el foco al aprendizaje profundo, que maneja mejor esos fondos complejos; entre ellos, los detectores en una etapa (familia YOLO, SSD) resultan más adecuados para la inferencia en tiempo real que los de dos etapas [12].

Una dificultad transversal de este problema es que las aeronaves deben detectarse a gran distancia, cuando ocupan pocos píxeles en la imagen. Este problema de detección de objetos pequeños ha motivado técnicas como el uso de *backbones* de alta resolución, el particionado de la imagen en fragmentos (*slicing*) [13], o el seguimiento (*tracking*) posterior mediante filtros de Kalman, que mantiene la detección entre fotogramas y descarta falsos positivos [14]. En una aplicación de seguridad, minimizar los falsos negativos es prioritario frente a los falsos positivos.

3.3. Conjuntos de datos

El principal obstáculo para entrenar estos modelos es la disponibilidad de datos reales etiquetados. El conjunto *Airborne Object Tracking* (AOT), publicado por Amazon Prime Air en AIcrowd [9] en el marco del *Airborne Object Tracking Challenge*, es hoy un recurso de referencia: aproximadamente cinco mil secuencias de vuelo de unos 120 segundos capturadas a 10 Hz, con millones de imágenes anotadas obtenidas mediante encuentros planificados

entre dos aeronaves con cámaras frontales. Su distribución bajo licencia CDLA-Permissive permite su uso académico y su volumen y variedad lo hacen adecuado para entrenar detectores sin necesidad de generar un conjunto propio. Es el conjunto elegido para este TFM.

Existen otros recursos con enfoques complementarios, como AVOIDDS [15], que además del conjunto anotado aporta un simulador de evaluación reproducible. Sin embargo, se ha optado por AOT por su mayor volumen de imágenes reales etiquetadas y por emplear exclusivamente capturas con cámara frontal (el mismo tipo de vista que ofrece la GoPro en este TFM).

3.4. Inferencia en el dispositivo

Ejecutar un modelo en tiempo real sobre un teléfono móvil impone restricciones de cómputo, memoria, consumo y latencia. Para afrontarlas se recurre a entornos de ejecución optimizados, cuantización de pesos y aceleradores hardware del propio dispositivo. LiteRT [8], anteriormente TensorFlow Lite, es el entorno de Google empleado en este trabajo, que permite llevar la inferencia al lado del dispositivo (*on-device*) sin depender de un servidor externo, con la ventaja añadida de no requerir conexión a Internet ni introducir latencia de red.

3.5. Soluciones existentes y posicionamiento

A continuación se presentan brevemente tres soluciones existentes representativas de distintos niveles de madurez y enfoques, para poder posicionar la propuesta de este TFM respecto a ellas.

3.5.1. Aerial Guard

Aerial Guard [4, 5] es una aplicación iOS orientada a la aviación ligera que emplea la cámara del propio teléfono móvil, colocado en la cabina, para detectar tráfico circundante mediante visión por computador. Su propuesta es notable por su bajo coste y facilidad de despliegue, si bien presenta dos limitaciones a las que responde este TFM: por un lado, la calidad de la imagen y el posicionamiento están condicionados por la ubicación del teléfono; por otro, el uso continuo de la cámara y de recursos del móvil dificulta simultáneamente

otras funciones útiles al piloto (navegación, radio, cartografía). La Figura 3.1 muestra la interfaz de Aerial Guard en operación.

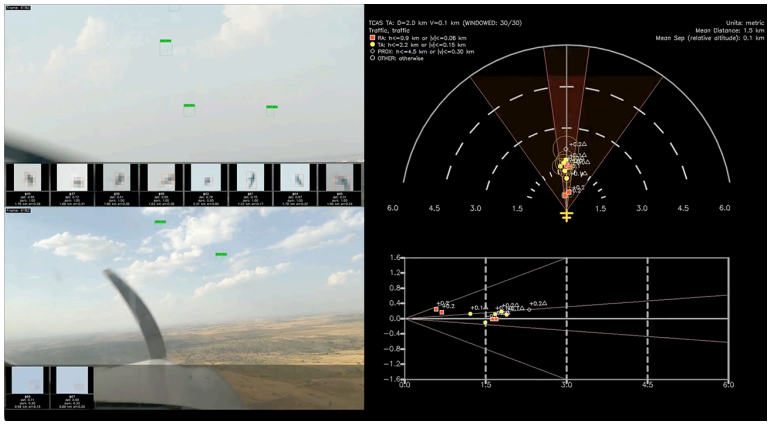


Figura 3.1: Interfaz de Aerial Guard mostrando la detección de una aeronave próxima sobre la cámara del móvil. Fuente: [5].

3.5.2. Iris Automation Casia

Iris Automation Casia [16] es un sistema comercial de *detect and avoid* orientado principalmente a aeronaves no tripuladas para operaciones BVLOS. Utiliza cámaras electroópticas dedicadas de gran angular y ejecuta la inferencia en una unidad de cómputo embarcada. Se trata de una solución madura y certificable, si bien su coste, peso y complejidad la sitúan fuera del alcance de la aviación ligera y ultraligera tripulada, escenario objetivo de este TFM. La Figura 3.2 muestra el módulo hardware de Casia.



Figura 3.2: Módulo Casia de Iris Automation, sistema comercial de *detect and avoid* para aeronaves no tripuladas. Fuente: [16].

3.5.3. AirTrack

AirTrack [14] es un sistema académico que aborda específicamente la detección de objetos aéreos pequeños a gran distancia mediante una combinación de detección basada en aprendizaje profundo y seguimiento multi-frame con filtros de Kalman. Los autores demuestran mejoras significativas en la reducción de falsos positivos y en el mantenimiento de la detección a lo largo del tiempo. Aunque su implementación de referencia no está pensada para ejecución embarcada en el dispositivo, sus técnicas de *tracking* temporal son directamente aplicables y se retoman como línea de mejora en el Capítulo 7. La Figura 3.3 resume el pipeline de detección y seguimiento de AirTrack.

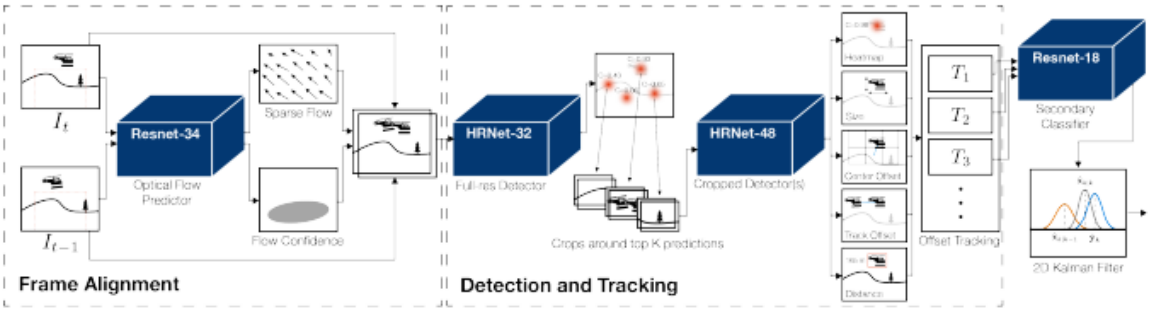


Figura 3.3: Pipeline de detección y seguimiento de AirTrack, combinando detección por aprendizaje profundo con seguimiento multi-frame mediante filtros de Kalman. Fuente: [14].

3.5.4. Posicionamiento de la propuesta

La propuesta de este TFM se sitúa en un espacio poco cubierto: sistema de detección visual embarcado sobre una plataforma Android estándar, con cámara externa GoPro dedicada (frente a Aerial Guard, que usa la cámara integrada), sin requerir cómputo especializado (frente a Iris Automation Casia), y con inferencia *on-device* en tiempo real mediante LiteRT (frente a AirTrack, no orientado a dispositivo móvil). El objetivo no es reemplazar los sistemas comerciales certificables, sino ofrecer una alternativa de bajo coste y accesible que pueda actuar como "ojo artificial" complementario para el piloto de aviación ligera.

4. Metodología de trabajo

En este capítulo se describe el método de trabajo seguido para el desarrollo del TFM, el marco tecnológico empleado, los recursos utilizados durante el proyecto y las principales limitaciones identificadas durante el mismo.

4.1. Enfoque metodológico

Debido al carácter experimental del proyecto, en el que intervienen múltiples componentes de distinta naturaleza como comunicación con hardware externo, procesamiento de vídeo, modelo de aprendizaje automático e interfaz de usuario, y como muchas decisiones técnicas no se podían dar por buenas sin probarlas antes, se ha optado por una **metodología iterativa e incremental** organizada en torno a *subhitos*: bloques de trabajo relativamente autocontenidos que aportan valor por sí mismos y que pueden validarse individualmente antes de integrarse con el resto.

Este enfoque presenta dos ventajas clave respecto a una metodología en cascada clásica. En primer lugar, permite descartar aproximaciones no viables en fases tempranas del proyecto sin comprometer el resto del proyecto: durante el desarrollo se han evaluado y descartado varias librerías de reproducción de vídeo en Android antes de adoptar la definitiva, decisión que se documenta en el Capítulo 5. En segundo lugar, ha permitido paralelizar el trabajo entre subhitos independientes, en particular el entrenamiento del modelo de detección (subhito de *Machine Learning*) y el desarrollo de la aplicación móvil (subhito de *Android*), optimizando el tiempo disponible.

Los subhitos principales en los que se ha organizado el proyecto son:

- **Subhito de comunicación con la cámara**: emparejamiento BLE, activación programática del punto de acceso WiFi, extracción de credenciales, unión del dispositivo Android a la red WiFi de la cámara y control por HTTP mediante la interfaz Open GoPro.
- **Subhito de *streaming* de vídeo**: recepción del flujo MPEG-TS sobre UDP, decodificación H.264 y presentación en la interfaz de la aplicación.
- **Subhito de *Machine Learning***: selección y preparación del conjunto de datos, entrenamiento del modelo YOLOv8n, evaluación mediante métricas objetivas y exportación al formato LiteRT.
- **Subhito de integración**: incorporación del modelo entrenado a la aplicación, muestreo de fotogramas del *stream*, ejecución de la inferencia y superposición de las detecciones en la interfaz.
- **Subhito de interfaz de usuario**: diseño visual del conjunto de la aplicación y elementos de apoyo a la interpretación de las detecciones (*Head-Up Display*, seguimiento visual y códigos de color).

Cada subhito ha sido validado individualmente antes de dar paso al siguiente. Los descartes técnicos y decisiones tomadas durante el proceso quedan explícitamente documentados en el Capítulo 5, con la intención de reflejar de forma honesta el camino real recorrido y las razones que lo justifican.

4.2. Planificación temporal

El proyecto se ha desarrollado a lo largo de aproximadamente seis meses, compaginándolo con actividad profesional a tiempo completo, con una dedicación media aproximada de siete a ocho horas semanales. La Figura 4.1 presenta el cronograma detallado de las fases del proyecto.

Como puede observarse, existe un solapamiento entre las fases del subhito de *Machine Learning* (entrenamiento del modelo) y las del subhito de comunicación con la cámara. Esto ha sido posible por su independencia técnica: mientras se realizaba entrenamiento del modelo (como se detalla en la Sección 5.3.4), se avanzaba en paralelo en el desarrollo de la aplicación Android.

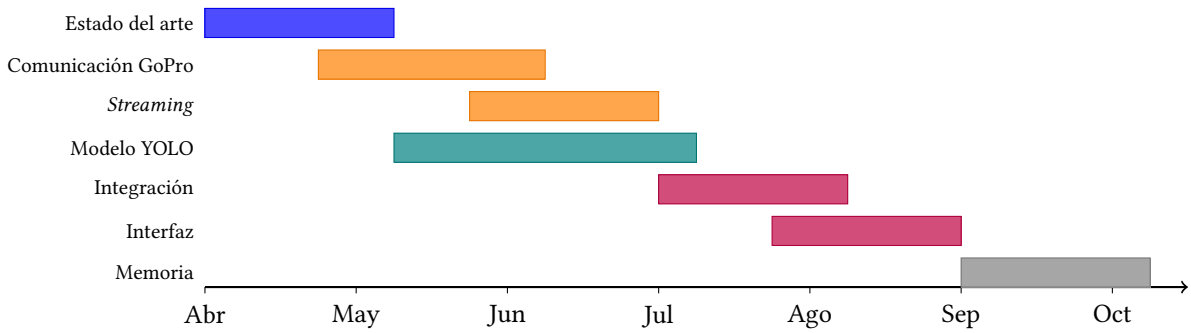


Figura 4.1: Cronograma de desarrollo del TFM organizado por subhitos. Las fases del desarrollo se solapan en el tiempo por su naturaleza incremental.

4.3. Marco tecnológico

El proyecto se apoya en un conjunto de tecnologías y herramientas que se agrupan a continuación por dominio.

4.3.1. Aplicación móvil

El desarrollo de la aplicación Android se ha realizado con **Android Studio Ladybug** como IDE, empleando **Kotlin** como lenguaje de programación y **Jetpack Compose**[1] como *framework* de interfaz de usuario. Se ha adoptado **Material Design 3** como sistema de diseño y componentes reutilizables.

Para la comunicación con la cámara GoPro se han empleado las APIs de **Bluetooth Low Energy**[2](BLE) del sistema Android, así como las de **WifiManager** y **ConnectivityManager** para el emparejamiento programático con el punto de acceso de la cámara. Las llamadas HTTP a la interfaz Open GoPro [6, 7] se han realizado mediante la librería **OkHttp**.

La reproducción del *stream* de vídeo se ha resuelto con **LibVLC 3.6.5** [17], tras evaluar y descartar otras alternativas. La inferencia del modelo de aprendizaje automático se realiza con **LiteRT** [8], cargando directamente el modelo desde los *assets* de la aplicación mediante `Interpreter`, sin depender de servicios en la nube.

[1]Kit de herramientas moderno de Android para construir interfaces declarativas, adoptado por Google como la vía recomendada para nuevas aplicaciones frente al sistema clásico basado en XML.

[2]Variante del estándar Bluetooth optimizada para dispositivos de bajo consumo, diseñada para comunicaciones cortas y poco frecuentes de bajo volumen.

4.3.2. Entrenamiento del modelo

Para la selección y preparación del conjunto de datos se ha empleado la **AWS Command Line Interface**[3] para la descarga desde el *bucket* S3 de Amazon Prime Air [9], y **Python 3** con las librerías **pandas**, **Pillow** y **Matplotlib** para el procesamiento del fichero de anotaciones y la conversión al formato requerido por YOLO. Todo este trabajo se ha realizado en un entorno local de **Jupyter Notebook**.

El entrenamiento del modelo se ha realizado con **Ultralytics YOLOv8n** [18] sobre **Google Colab Pro**, aprovechando una GPU **NVIDIA L4**[4]. La exportación al formato LiteRT (`.tflite`) se ha realizado con el propio conversor de Ultralytics. Los pesos entrenados se han respaldado en **Google Drive** mediante *callbacks* tras cada época para prevenir pérdidas por desconexión de la sesión.

4.3.3. Control de versiones y colaboración

El código fuente se aloja en un repositorio privado de **GitHub**, empleando **Git** para el control de versiones. La redacción de la presente memoria se ha realizado con **LaTeX** sobre la plataforma **Overleaf**, compilando con **XeLaTeX** y utilizando la plantilla oficial de la ESI/ESII de la Universidad de Castilla-La Mancha.

La Figura 4.2 resume el marco tecnológico completo agrupado por dominio.

4.4. Recursos utilizados

Los recursos empleados durante el desarrollo del proyecto son los siguientes.

4.4.1. Recursos hardware

- **Cámara**: GoPro Hero 12 Black, con firmware v01.30 y compatibilidad con Open GoPro [7]. Se ha utilizado el modo *streaming* en calidad 1080p a 30 Hz.
- **Dispositivos Android de prueba**: dos dispositivos Xiaomi Redmi. La aplicación se ha desarrollado principalmente sobre un *Redmi Note 8* (Android 11, procesador MediaTek Helio G85) y la validación funcional final se ha realizado sobre un *Redmi Note 10*

[3] Los datos del AOT se distribuyen desde un *bucket* S3 público en modo *no-sign-request*.

[4] Aceleradora de la generación Ada Lovelace de NVIDIA, orientada a cargas de inferencia y *fine-tuning* en la nube, con 24 GB de memoria dedicada.

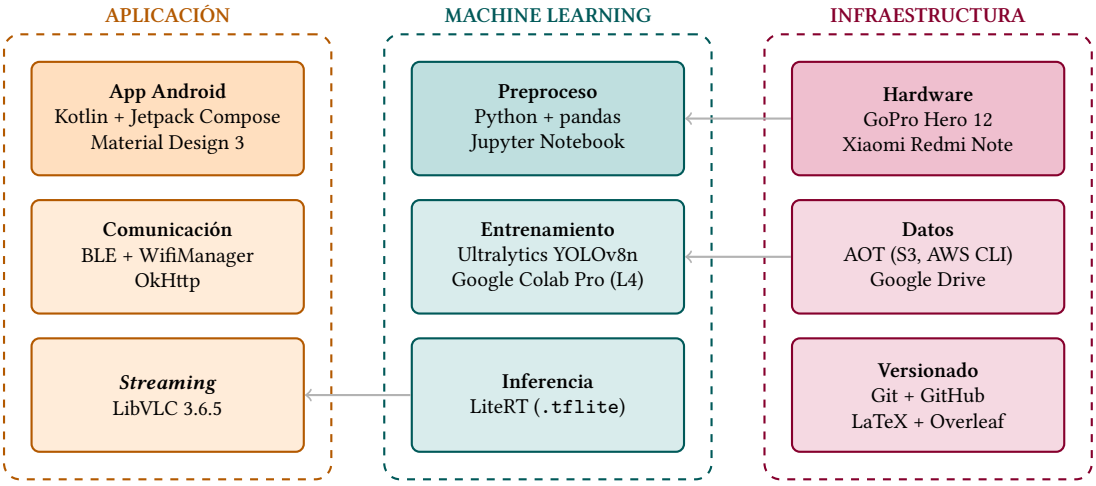


Figura 4.2: Marco tecnológico del proyecto organizado por dominios funcionales. Las flechas indican dependencias significativas entre dominios.

(Snapdragon 678). La utilización de ambos dispositivos ha sido determinante para la identificación de limitaciones del pipeline gráfico específicas del hardware, tal como se documenta en el Capítulo 5.

- **Estación de trabajo local**: MacBook Pro con arquitectura Apple Silicon, empleado para el desarrollo de la aplicación, el preprocesado del *dataset* y la redacción de la memoria.
- **GPU en la nube**: NVIDIA L4 (24 GB) para el entrenamiento del modelo, accedida a través del servicio Google Colab.

4.4.2. Recursos software y servicios en la nube

Se ha empleado software libre y servicios en la nube en la mayoría de los componentes del proyecto. Los servicios utilizados han sido **Google Colab Pro** para acceder a GPUs de alto rendimiento con sesiones prolongadas sin desconexiones, y **Google One** para almacenar el *dataset* preprocesado y los pesos del modelo entrenado.

4.5. Limitaciones

Durante el desarrollo del proyecto se han identificado una serie de limitaciones que han condicionado ciertas decisiones y que conviene explicitar:

- **Tiempo de dedicación**: la compatibilización del TFM con actividad profesional ha sido una limitación para la dedicación semanal.

- **Imposibilidad de captura real en vuelo**: no se dispone de acceso operativo a una aeronave para capturar situaciones reales de aproximación entre tráficos, motivo por el cual se ha recurrido a un *dataset* público existente (AOT).
- **Alcance del *dataset***: por razones de coste temporal de descarga y procesamiento se ha trabajado con un subconjunto de cinco secuencias del AOT (aproximadamente 6000 imágenes), lo cual es suficiente para validar la viabilidad de la propuesta pero no para explotar el máximo del modelo. La ampliación del conjunto de entrenamiento queda como línea de trabajo futuro.
- **Compatibilidad hardware**: se ha detectado una incompatibilidad específica entre el compositor gráfico del dispositivo Xiaomi Redmi Note 8 con HyperOS y la lectura de fotogramas del reproductor LibVLC, que ha condicionado la validación funcional final al Xiaomi Redmi Note 10. Este hecho se documenta en el Capítulo 5.

5. Detección de amenazas aéreas mediante visión por computador

En este capítulo se describe el desarrollo del sistema siguiendo el flujo temporal real de desarrollo del proyecto, agrupado en cinco bloques que se corresponden con los subhitos definidos en el Capítulo 4: la comunicación con la cámara GoPro (Sección 5.1), el *pipeline* de *streaming* de vídeo (Sección 5.2), el modelo de detección (Sección 5.3), la integración del modelo con la aplicación (Sección 5.4) y el diseño de la interfaz de usuario (Sección 5.5).

Para ofrecer una visión realista del proceso llevado a cabo, se han documentado las distintas opciones que fueron evaluadas y descartadas, así como las razones que justifican las decisiones tomadas. Los fragmentos de código que se han considerado más importantes se han incluido en el Anexo A.

5.1. Comunicación con la cámara

La aplicación necesita comunicarse con la cámara GoPro Hero 12 Black en tres niveles: descubrimiento y emparejamiento inicial mediante BLE, establecimiento de una red WiFi de datos punto a punto para el flujo de vídeo, y control por HTTP de las funciones de la cámara relevantes para la aplicación. Para ello se ha hecho uso de la interfaz *Open GoPro* [6, 7], especificación publicada por GoPro para la integración de sus cámaras con aplicaciones de terceros.

La Figura 5.1 resume el flujo temporal completo desde el descubrimiento hasta la recepción del primer fotograma decodificado.

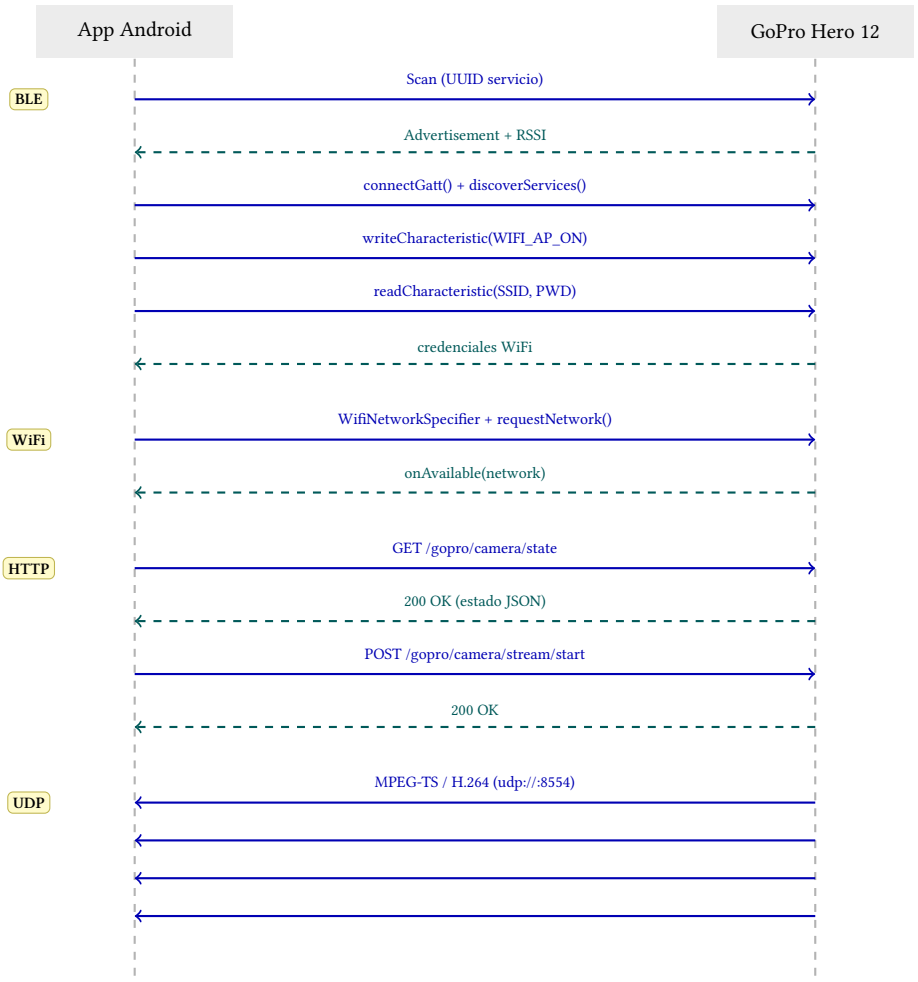


Figura 5.1: Diagrama de secuencia del establecimiento de la comunicación con la GoPro, desde el descubrimiento BLE hasta la recepción del flujo de vídeo.

5.1.1. Emparejamiento y control por Bluetooth Low Energy

La cámara publica un servicio BLE identificable por un UUID[1] definido en la especificación Open GoPro, que la aplicación utiliza para descubrirla. El proceso se ha implementado con las APIs nativas de Android (`BluetoothLeScanner`, `BluetoothGatt`) y se ha organizado en dos componentes dentro del módulo `gopro`: uno que descubre las cámaras cercanas filtrando por dicho UUID, y otro que gestiona la conexión GATT[2] una vez establecida.

[1] *Universally Unique Identifier*: identificador de 128 bits que, por su longitud, se puede generar sin coordinación central con la garantía práctica de que no colisione con otro, lo que permite nombrar de forma inequívoca servicios y características BLE en todo el ecosistema Bluetooth.

[2] *Generic Attribute Profile*: perfil que define cómo dos dispositivos BLE intercambian datos organizados en servicios y características.

Un aspecto que requirió especial atención fue el manejo de las *Client Characteristic Configuration Descriptors* (CCCD): para poder recibir notificaciones asíncronas de la cámara (por ejemplo, la confirmación del disparador o el cambio de estado del punto de acceso WiFi), es necesario habilitar explícitamente las notificaciones tanto en el cliente (mediante `setCharacteristicNotification`) como en el servidor (escribiendo el valor `ENABLE_NOTIFICATION_VALUE` en el descriptor CCCD de la característica), operación cuya omisión es una fuente común de errores silenciosos en desarrollo BLE.

También se detectó una particularidad de la Hero 12 respecto a documentación de modelos anteriores: la cámara expone dos servicios BLE distintos, uno para control (`0000fea6-...`) y otro para configuración WiFi (`b5f90001-...`). Ambos deben descubrirse tras la conexión, y los comandos y lecturas se distribuyen entre ellos según su naturaleza. La clase GoProUuids del módulo contiene la lista de UUIDs utilizados.

5.1.2. Activación del punto de acceso y conexión WiFi

Una vez establecido el canal BLE de control, la aplicación solicita a la cámara el arranque de su punto de acceso WiFi[3] mediante un comando específico. Tras la confirmación se leen por BLE las credenciales (SSID y contraseña) y se conecta el dispositivo a dicha red.

La conexión programática al WiFi de la cámara desde Android 10 o superior requiere el uso de la API `WifiNetworkSpecifier`, que se solicita a través del gestor de conectividad del sistema (`ConnectivityManager`) mediante una llamada a `requestNetwork()`. Un punto crítico es que, aunque el dispositivo esté conectado a la red WiFi de la cámara, por defecto el resto de aplicaciones y componentes (incluyendo la librería HTTP `OkHttp` y el reproductor LibVLC) siguen usando la red principal del teléfono. Para asegurar que las peticiones HTTP y el *stream* de vídeo viajen por la red de la cámara es necesario indicar explícitamente al sistema, mediante una llamada a `bindProcessToNetwork()`, que la aplicación debe usar como red por defecto la que se acaba de recibir en el aviso de conexión establecida (*callback* `onAvailable`). Esta llamada requiere además el permiso `CHANGE_NETWORK_STATE` declarado en el manifiesto de la aplicación.

Además, cuando el dispositivo está conectado al punto de acceso de la cámara, esta es accesible en la dirección local `10.5.5.9` mediante peticiones HTTP sin cifrar (sin TLS). Como

[3]Red WiFi propia que la cámara emite en modo *Access Point* (AP), a la que se conecta el dispositivo Android para recibir el flujo de vídeo.

Android bloquea por defecto este tipo de conexiones desde su versión 9, ha sido necesario configurar un fichero `network_security_config.xml` en la aplicación que autoriza explícitamente el tráfico HTTP hacia esa dirección concreta, sin relajar la política de seguridad para el resto de comunicaciones de la aplicación.

5.1.3. Control por HTTP mediante la interfaz Open GoPro

Con la conexión WiFi establecida, la aplicación se comunica con la cámara mediante HTTP. Se ha implementado un componente `GoProHttpClient` basado en `OkHttp`, configurado para utilizar el `socketFactory` del `Network` de la cámara (obtenido en la conexión WiFi), de forma que las peticiones se cursen efectivamente por ese enlace y no por la red por defecto del dispositivo.

Las llamadas relevantes para el sistema son dos: la consulta del estado de la cámara (`GET /gopro/camera/state`), útil para verificar la disponibilidad y el modo activo, y el arranque del *stream* de *preview*[4] (`POST /gopro/camera/stream/start`), que solicita a la cámara la emisión del flujo de vídeo en formato MPEG-TS sobre UDP al puerto 8554 del dispositivo conectado. Existe también la llamada de parada correspondiente (`/gopro/camera/stream/stop`), invocada por la aplicación al detener el *streaming* o al desconectarse.

La respuesta al arranque del *stream* no contiene datos inmediatamente útiles, ya que el flujo real llegará por el canal UDP separado. Sin embargo, se emplea la respuesta HTTP para confirmar que la cámara ha aceptado la solicitud y como *feedback* para la interfaz de usuario.

5.2. Pipeline de *streaming* de vídeo

Una vez establecida la comunicación con la cámara y arrancado el *stream* en el lado de la GoPro, el objetivo del lado de la aplicación Android es recibir, decodificar y mostrar en pantalla el flujo de vídeo MPEG-TS sobre UDP, y hacer accesibles los fotogramas decodificados para su posterior análisis por el modelo de detección. En concreto, el *preview stream* de la GoPro proporciona vídeo H.264 a unos 854×480 px y ~ 30 fps, resolución

[4]Flujo de vídeo de baja latencia y calidad reducida que la cámara emite en tiempo real, pensado para previsualización en apps de control; no debe confundirse con la grabación en alta resolución que la cámara almacena internamente en su tarjeta de memoria.

pensada para supervisión en tiempo real y suficiente para la detección a bordo.

Este objetivo ha resultado ser el bloque técnicamente más complejo del proyecto. Antes de llegar a la solución se evaluaron tres opciones distintas, cada una con sus propios problemas. Se documentan aquí las tres en el mismo orden en el que se probaron, porque las razones por las que se descartó cada una explican por qué se acabó eligiendo la siguiente, y forman parte del aprendizaje técnico del TFM.

5.2.1. Evaluación de alternativas descartadas

Google Media3 / ExoPlayer

La primera opción evaluada fue **Google Media3** (evolución moderna de ExoPlayer), la solución que Google recomienda oficialmente para reproducir vídeo en Android. Media3 permite alimentar el reproductor con una fuente de datos personalizada, como el flujo MPEG-TS que envía la GoPro por UDP.

Se desarrolló una fuente a medida que abría el socket UDP en el puerto 8554, recogía los paquetes de la cámara y los entregaba al reproductor. Aunque arrancaba, la reproducción fallaba con un error al mostrar los fotogramas en los dispositivos Xiaomi utilizados. Tras varios ajustes (distintos tipos de vista, parámetros de reproducción y forzado del decodificador por software), se descartó la vía por una incompatibilidad de hardware.

Pipeline propio con MediaCodec

La segunda opción consistió en construir el *pipeline*[5] de vídeo completo desde cero, empleando únicamente las herramientas nativas de Android: un módulo que recibe los paquetes de la cámara, otro que extrae de ellos el vídeo H.264, y un decodificador de bajo nivel (`MediaCodec`) que se encarga de convertir esa información en fotogramas visibles.

Esta vía ofrece control total sobre el proceso, pero introduce una gran complejidad: hay que ordenar manualmente los paquetes que llegan por la red, sincronizar los tiempos de cada fotograma para que se muestren en el instante correcto, alimentar el decodificador al ritmo adecuado y manejar sus distintos estados internos con mucho cuidado.

[5]Cadena de procesamiento por la que pasa el vídeo desde que llega a la aplicación hasta que se muestra en pantalla: recepción, decodificación, presentación.

Tras varios días de trabajo la implementación llegó a mostrar vídeo en pantalla, pero con problemas recurrentes de estabilidad: el decodificador se bloqueaba pasados unos segundos y la aplicación se colgaba al cambiar el destino de la reproducción. La conclusión fue que reimplementar lo que las librerías ya resuelven no aportaba valor al proyecto y consumía tiempo que era más productivo invertir en los otros bloques del sistema.

5.2.2. Adopción de LibVLC

La tercera opción, y finalmente la adoptada, fue **LibVLC 3.6.5** [17], la biblioteca multimedia del reproductor VLC[6], que ofrece en Android tanto un componente reproductor como una vista lista para mostrar el vídeo.

La implementación es notablemente más sencilla: basta con indicar a LibVLC la dirección de red por donde llega el vídeo (`udp://@:8554`), asignarle la vista de destino y arrancar la reproducción. LibVLC se encarga internamente de reordenar los paquetes, elegir el mejor decodificador disponible, sincronizar los tiempos y mostrar cada fotograma. Toda la complejidad que antes había que gestionar a mano queda resuelta por la biblioteca, con un resultado estable en los dispositivos probados. Además, los parámetros de arranque se ajustaron para minimizar la latencia: los tres *buffers* internos de LibVLC (red, transmisión en directo y almacenamiento local) se configuraron a 100 ms, reduciendo el retraso entre lo que graba la cámara y lo que ve el piloto.

5.2.3. Captura de fotogramas para inferencia

El objetivo final del *pipeline* no es solo mostrar el vídeo en pantalla, sino capturar fotogramas periódicamente para pasárselos al modelo de detección. Este bloque presentó problemas específicos de compatibilidad con el hardware.

El primer intento empleó la API estándar de Android para copiar el contenido de la superficie de vídeo a una imagen en memoria (`Bitmap`), pero la llamada devolvía un error indicando que no había fotogramas disponibles. Al analizar el comportamiento se descubrió que LibVLC, cuando usa el decodificador hardware, dibuja los fotogramas directamente sobre la superficie de vídeo por una vía interna que no pasa por el mecanismo

[6] Se seleccionó la versión 3.6.5 y no la rama 4.x preliminar por su mayor estabilidad y compatibilidad con las plataformas Android objetivo.

estándar del que depende esa API.

Como alternativa se configuró LibVLC para usar una `TextureView`, un tipo de superficie que sí permite leer los fotogramas renderizados desde su memoria gráfica. Esta segunda aproximación fue la definitiva.

Cabe destacar que la sustitución no resolvió el problema en el dispositivo original de desarrollo (Xiaomi Redmi Note 8 con procesador MediaTek), donde la captura seguía siendo intermitente. La validación funcional final se realizó sobre un Xiaomi Redmi Note 10 con procesador Snapdragon, donde funciona de forma estable. La limitación del Note 8 parece relacionada con la gestión del vídeo de HyperOS sobre chipsets MediaTek de gama de entrada, y se documenta como consideración de compatibilidad para futuras implantaciones.

La Figura 5.2 resume el *pipeline* completo de *streaming* adoptado, desde la recepción del vídeo hasta la disponibilidad del fotograma para el modelo.

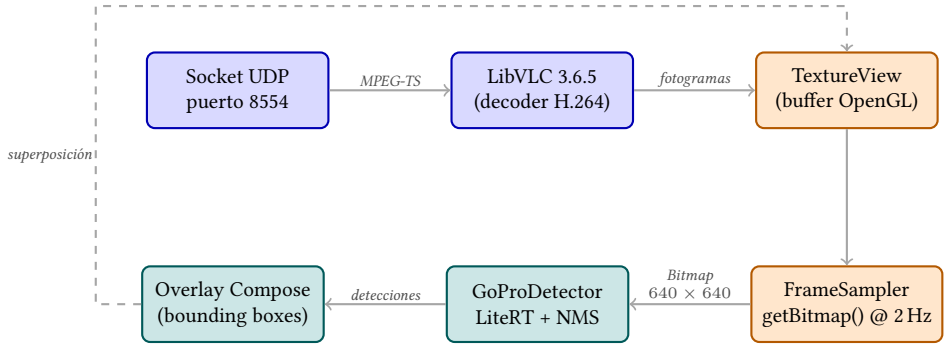


Figura 5.2: Pipeline final de recepción del *stream*, decodificación, captura periódica de fotogramas e inferencia con superposición de resultados en la vista.

El componente encargado del muestreo (`FrameSampler`) captura fotogramas a una frecuencia configurable, por defecto 2 veces por segundo, valor que se justifica en la Sección 5.4. Incluye además un mecanismo de control para evitar solapar inferencias en el caso de que el modelo tarde más de lo previsto en procesar un fotograma: mientras hay una inferencia en curso, los nuevos fotogramas se descartan silenciosamente. La implementación completa se recoge en el Anexo A.

5.3. Modelo de detección

Esta sección describe el subhito de *Machine Learning*: la selección y preparación del conjunto de datos, el entrenamiento del modelo y su exportación al formato ejecutable en dispositivo, documentando las iteraciones y decisiones tomadas.

5.3.1. Conjunto de datos AOT: descarga y análisis

El conjunto *Airborne Object Tracking* (AOT) [9] está compuesto por aproximadamente cinco mil secuencias de vuelo capturadas por aeronaves reales equipadas con cámaras frontales, publicado por Amazon Prime Air bajo licencia CDLA-Permissive y distribuido a través de un *bucket* S3 público. El *dataset* ocupa varios terabytes y contiene unos 5-6 millones de imágenes, por lo que trabajar con todos quedaba fuera del alcance del TFM. Se optó por seleccionar un subconjunto reducido pero representativo.

El *dataset* incluye un fichero maestro (`groundtruth.csv`, 311 MB) que enumera las anotaciones con el formato `gt_left, gt_top, gt_right, gt_bottom` en coordenadas absolutas de píxel, sobre imágenes PNG en escala de grises de 2448×2048. Un segundo fichero (`valid_encounters.csv`) resume metadatos por secuencia: identificador de vuelo, categoría del objeto (aeronave o pájaro), posición respecto al horizonte (*above horizon* o *below horizon*), distancia mínima al objeto y número de frames. La Figura 5.3 resume el formato de anotaciones del AOT.

Se seleccionaron cinco secuencias buscando equilibrio entre condiciones de vuelo: dos secuencias *above horizon* (fondo cielo, más simple de detectar) y tres *below horizon* (fondo con *clutter* de terreno, escenario más difícil), con rangos de distancia distintos (entre 164 m y 458 m). El subconjunto resultante contiene 5995 fotogramas y 6247 anotaciones. La descarga selectiva desde S3 se realizó mediante `aws s3 cp` en modo `--no-sign-request`, filtrando por los identificadores de vuelo seleccionados. La Figura 5.4 muestra fotogramas de tres de estas secuencias, donde se aprecia la variedad de condiciones presentes en el subconjunto seleccionado.

El análisis inicial del subconjunto reveló la distribución de anotaciones por categoría de objeto que se muestra en la Figura 5.5.

Dada la naturaleza del problema, se tomó la decisión de fusionar las categorías *Airplane1* y *Helicopter1* en una única clase `aircraft` (5981 anotaciones), y mantener `bird`

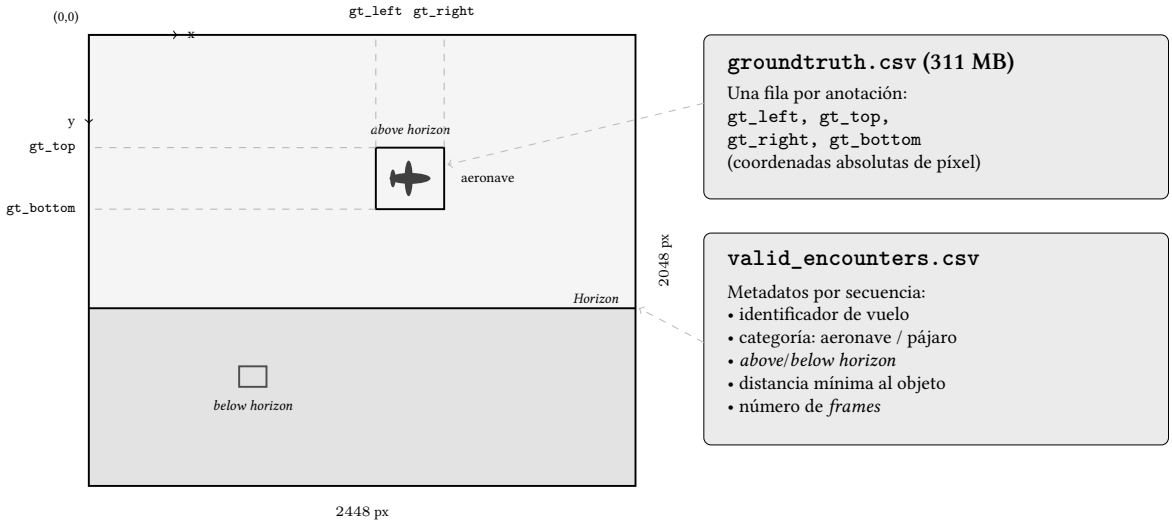


Figura 5.3: Formato de las anotaciones del *dataset* AOT: coordenadas de la *bounding box* sobre el fotograma y metadatos por secuencia. Imagen PNG en escala de grises de 2448×2048.



(a) *Above horizon* (b) *Below horizon* (c) Contraluz

Figura 5.4: Fotogramas representativos de tres de las cinco secuencias del *dataset* AOT empleadas, ilustrando la variedad de condiciones de vuelo: (a) escenario *above horizon*, con cielo despejado y buena visibilidad; (b) escenario *below horizon*, con aeronave pequeña sobre paisaje con niebla y *clutter* de terreno; (c) escenario con contraluz intenso, que reduce el contraste local y dificulta la detección. El *dataset* original es en escala de grises, lo que refuerza la dependencia del contraste con el fondo.

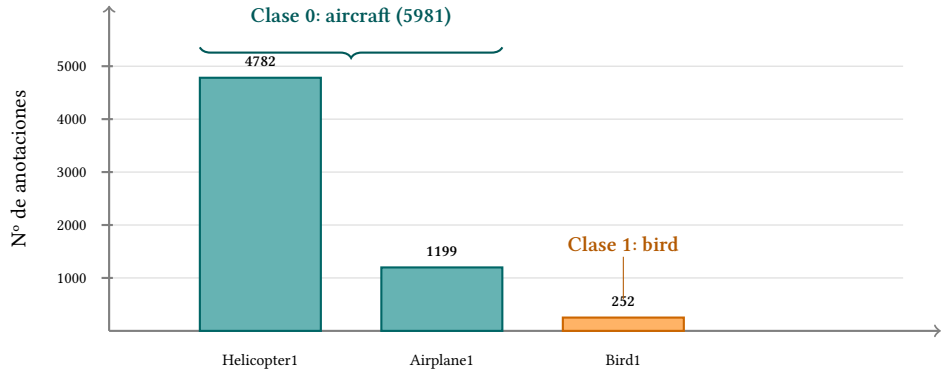


Figura 5.5: Distribución de anotaciones por categoría de objeto en el subconjunto AOT. Las categorías *Airplane1* y *Helicopter1* se fusionan en una única clase *aircraft* para el entrenamiento.

como clase separada (252 anotaciones). Se asume que el sistema no necesita distinguir tipo de aeronave, y la fusión aumenta el número de ejemplos por clase mejorando la capacidad de generalización del modelo. La clase `bird` se conserva como categoría independiente aunque desbalanceada, para reducir falsos positivos sobre objetos aéreos que no representan riesgo comparable al de otra aeronave.

Adicionalmente, se descartaron 14 anotaciones con identificador de objeto ausente (`NaN`) que aparecían en el subconjunto y que corresponden a errores puntuales del *dataset* original.

5.3.2. Preparación del subconjunto y formato YOLO

El formato de anotaciones esperado por Ultralytics YOLOv8 difiere del utilizado por AOT: cada imagen debe acompañarse de un fichero de texto `.txt` homónimo, con una línea por objeto anotado en formato `class x_center y_center width height`, donde las cuatro coordenadas se expresan normalizadas al intervalo $[0, 1]$ respecto a las dimensiones de la imagen. Se implementó un *notebook* en Python que realiza la conversión, agrupa las anotaciones por imagen y estructura los ficheros en el árbol de directorios estándar de YOLO (`images/train`, `images/val`, `labels/train`, `labels/val`). La Figura 5.6 muestra la conversión al formato esperado por YOLOv8.

Una vez convertidas las anotaciones, un análisis de las coordenadas normalizadas confirmó la naturaleza específica de este problema de detección: el ancho medio de los *bounding boxes* es de 0.0125 (aproximadamente 30 píxeles en la resolución original de

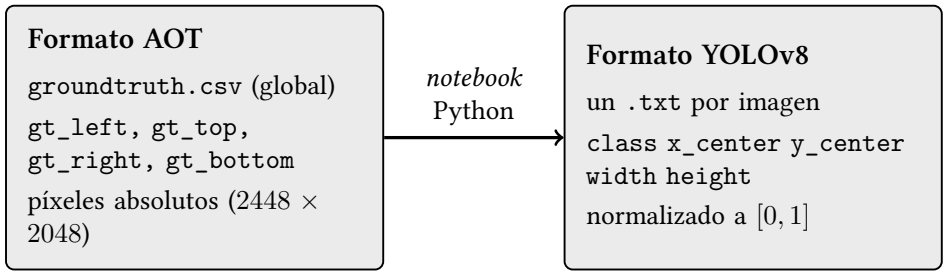


Figura 5.6: Conversión del formato de anotaciones de AOT (píxeles absolutos) al esperado por Ultralytics YOLOv8 (coordenadas normalizadas y un fichero por imagen).

2448), y el alto medio de 0.0115 (aproximadamente 24 píxeles). En términos absolutos, los objetos a detectar son **diminutos**, situándose en el rango que la literatura de visión por computador clasifica como *tiny objects*. Esta característica es la que justifica la decisión posterior de entrenar el modelo a resolución 1280×1280 en lugar de la habitual 640×640: reducir demasiado la imagen colapsaría los objetos hasta tamaños en los que el detector no puede aprender características discriminativas.

Sobre la posición vertical, el rango de y_center está acotado entre 0.39 y 0.71, lo que confirma que las aeronaves aparecen mayoritariamente en la banda central de la imagen, alineadas con la línea del horizonte. Es coherente con la naturaleza del *dataset* y con el escenario airprox real. El análisis de las coordenadas normalizadas se resume en la Figura 5.7.

5.3.3. División train/val: primera iteración y corrección

La primera iteración de división del *dataset* adoptó una estrategia **por secuencia**: cuatro secuencias completas para entrenamiento (4782 imágenes) y una secuencia entera a validación (1199 imágenes). Con ello el modelo se valida sobre una secuencia no vista durante el entrenamiento, evitando la contaminación temporal entre fotogramas contiguos. Cabe señalar que la única secuencia con anotaciones de la clase `bird` (Bird1) forma parte del conjunto de entrenamiento; por tanto, en esta iteración `bird` no está representada en validación, lo que impide obtener métricas fiables para ella.

Con esta división, el entrenamiento de 50 épocas produjo un resultado en el que las métricas de entrenamiento (*box loss*, *classification loss*) descendieron con normalidad, indicando que el modelo aprendía, pero las de validación (*mAP*, precisión, *recall*) permane-

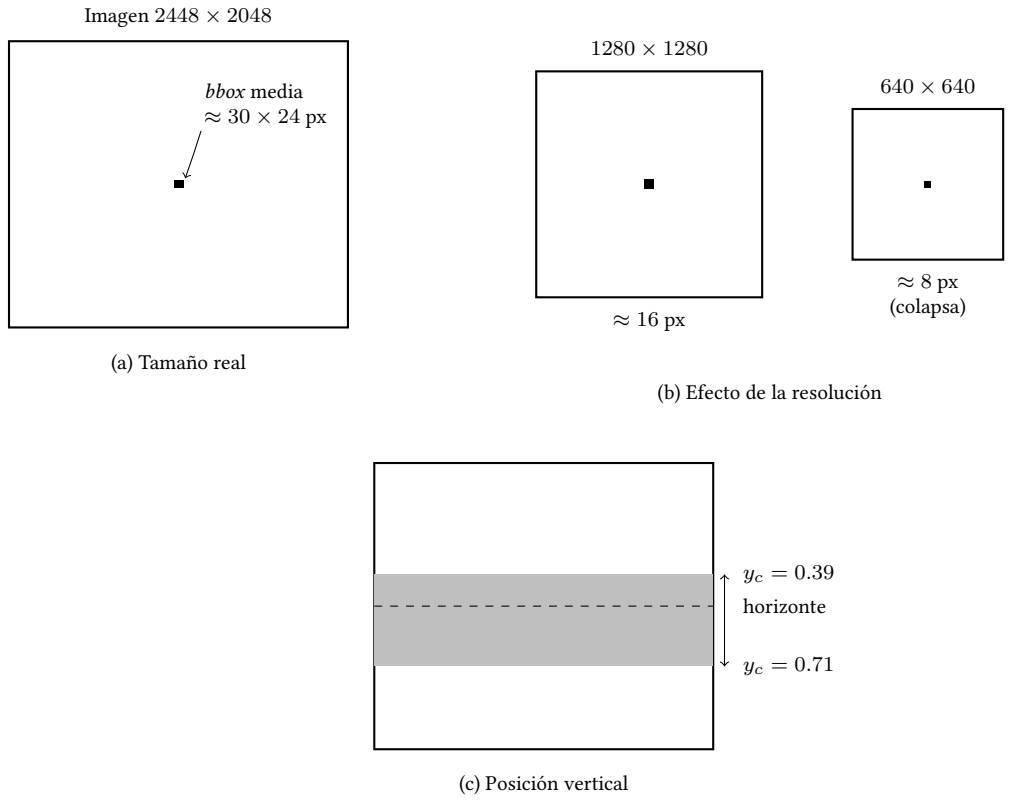


Figura 5.7: Análisis de las coordenadas normalizadas. (a) El tamaño medio de las cajas ($\approx 0.0125 \times 0.0115$) las clasifica como *tiny objects*. (b) YOLO redimensiona las imágenes a formato cuadrado; a 640×640 el objeto colapsa a la mitad de píxeles, lo que justifica entrenar a 1280×1280. (c) El centro vertical se concentra en torno al horizonte.

cieron en cero.

El análisis reveló la causa: siendo cada secuencia un vuelo distinto con su propia iluminación, tipo de aeronave, meteorología y fondo, el modelo aprendía los objetos concretos de las cuatro secuencias de entrenamiento sin generalizar a la quinta. Es un caso claro de *overfitting* por una división con muy poca diversidad.

En la segunda iteración se pasó a una división **por fotograma estratificada**: se mezclan las cinco secuencias y se reparte aleatoriamente el 80 % de los fotogramas para entrenamiento y el 20 % para validación, con semilla fija (`seed=42`) para garantizar reproducibilidad. La Figura 5.8 muestra ambas estrategias.

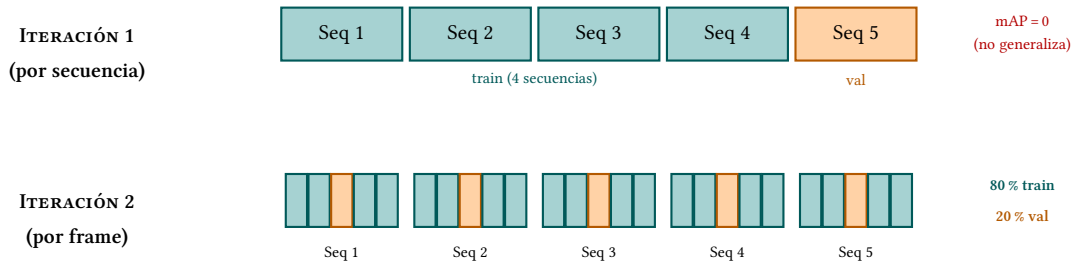


Figura 5.8: Comparación de las dos estrategias de división del *dataset*. La primera (por secuencia) no generaliza; la segunda (por frame estratificada) reparte todas las secuencias entre *train* y *val*.

Con esta división, el entrenamiento produjo métricas de validación que subían de forma sostenida desde la primera época, confirmando que el problema era la falta de diversidad y no un fallo del proceso de entrenamiento. La distribución final resultante fue de 4784 imágenes de *train* y 1197 imágenes de *val*.

5.3.4. Entrenamiento del modelo

Como arquitectura base se eligió **YOLOv8n** (variante *nano*), la más pequeña de la familia YOLOv8 de Ultralytics [18], con unos 3 millones de parámetros y 8.2 GFLOPs. Se eligió esta variante porque el objetivo es que el modelo funcione en un dispositivo móvil, y las variantes mayores (*small*, *medium*, *large*), aunque más precisas, son demasiado pesadas para ir en tiempo real en un *smartphone* de gama baja. Además, se aplicó *transfer learning*[7] a partir de los pesos preentrenados en COCO (80 clases genéricas), remapeando la capa final a las dos clases del proyecto, algo que Ultralytics gestiona automáticamente mediante el parámetro `nc=2` inferido del `dataset.yaml`.

Los hiperparámetros de entrenamiento adoptados fueron los siguientes:

- **Épocas**: 25 (tras verificar en un entrenamiento inicial de 50 épocas que el modelo convergía sobradamente antes de la 25).
- **Resolución de imagen**: 1280×1280 (justificada en la Sección 5.3.2).
- ***Batch size***: 8 (limitado por la memoria de la GPU).
- **Optimizador**: AdamW con *learning rate* inicial 0.001667, seleccionado automáticamente por Ultralytics.

[7] Técnica que consiste en partir de un modelo ya entrenado en una tarea distinta, en lugar de entrenarlo desde cero, para aprovechar lo que ya ha aprendido y así necesitar menos datos y tiempo de entrenamiento.

- ***Early stopping***: `patience=8` épocas sin mejora.

El entrenamiento se ejecutó sobre Google Colab con GPU NVIDIA L4 (24 GB), completando las 25 épocas en aproximadamente 1 hora y 40 minutos.

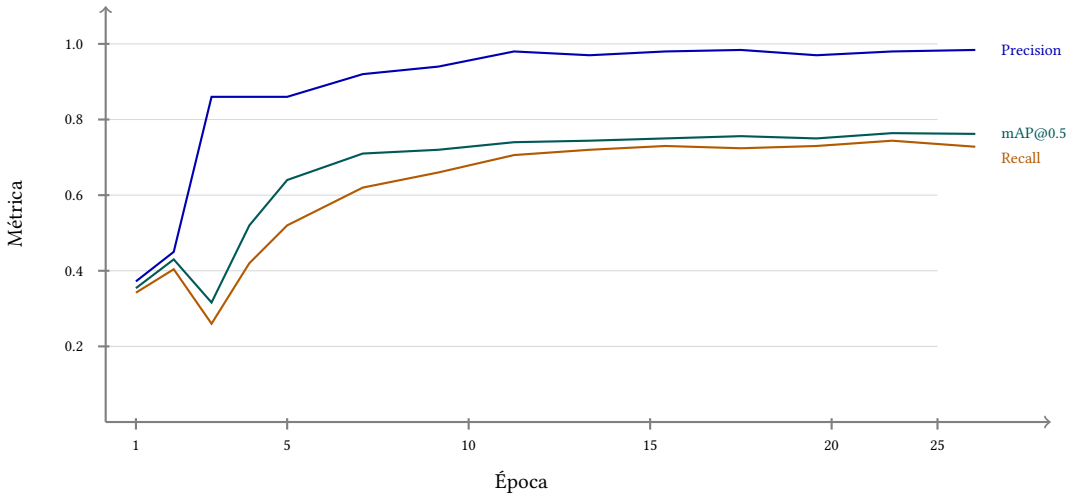


Figura 5.9: Evolución de las métricas de validación (Precision, Recall y mAP@0.5) sobre el conjunto de validación durante las 25 épocas del entrenamiento.

Como muestra la Figura 5.9, el modelo converge rápido: en unas 10 épocas alcanza el 90 % de su rendimiento final, y desde la época 15 las métricas se estabilizan. La *precision*, muy alta desde la época 10 (por encima de 0.95), refleja que las detecciones son casi siempre correctas.

5.3.5. Métricas finales y decisiones sobre el modelo

Las métricas finales del mejor modelo (`best.pt`, seleccionado automáticamente por Ultralytics como la época con mayor *mAP* global) sobre el conjunto de validación se recogen en la Tabla 5.1.

Los resultados por clase muestran comportamientos claramente distintos:

- **`aircraft`**: buen rendimiento, con $mAP@0.5 = 0.993$ y $recall = 0.982$, por encima de lo necesario para un primer sistema de aviso visual al piloto.
- **`bird`**: con solo 252 anotaciones en entrenamiento y 46 en validación, presenta un *recall* bajo (0.47), que se atribuye al desbalance de clases.

Clase	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Global (*all*)	0.984	0.728	0.762	0.481
`aircraft`	0.967	0.982	**0.993**	0.663
`bird`	1.000	0.474	0.530	0.299

Tabla 5.1: Métricas de validación del modelo `best.pt` sobre 1197 imágenes de validación (1197 instancias de `aircraft` y 46 de `bird`).

En el contexto de la aplicación, esta limitación en `bird` es aceptable, dado que la detección de aves no es el objetivo principal del sistema, sino que su rol es fundamentalmente reducir falsos positivos sobre objetos aéreos que no se consideran en este trabajo, al representar un riesgo menor. Su mejora queda como línea de trabajo futuro.

La inferencia durante la validación tardó 3.5 ms por imagen en la GPU L4 (0.5 ms de preprocesado y 0.6 ms de postprocesado), unos 5 ms en total por fotograma. Este dato no es directamente extrapolable al móvil, donde la inferencia corre en CPU o con el delegado XNNPACK y la latencia sube a decenas o incluso cientos de milisegundos, como se detalla en el Capítulo 6.

5.3.6. Exportación al formato LiteRT

Ultralytics proporciona un exportador integrado que convierte el modelo entrenado (`.pt`) al formato LiteRT (`.tflite`) mediante `model.export(format='tflite', imgsz=640)`. Se optó por exportar a resolución 640×640 en lugar de la resolución de entrenamiento (1280×1280) para reducir el coste de inferencia en el dispositivo móvil, asumiendo la degradación de precisión. El resultado es un archivo que ocupa aproximadamente 12 MB en precisión FP32 y se incluye directamente en los *assets* de la aplicación Android.

El modelo exportado espera un tensor de entrada `[1, 3, 640, 640]` en `float32`, es decir, en formato CHW (*channels, height, width*), típico de PyTorch pero distinto del formato HWC (*height, width, channels*) que utilizan habitualmente LiteRT y las APIs gráficas de Android. Esta diferencia obligó a adaptar el preprocesado en la aplicación, como se explica en la Sección 5.4. El tensor de salida es `[1, 6, 8400]`: 8400 posibles detecciones, cada una con 6 valores $(cx, cy, w, h, score_{aircraft}, score_{bird})$, todos normalizados entre 0 y 1 respecto al tamaño de entrada.

5.4. Integración modelo-aplicación

Esta sección describe cómo el modelo entrenado y exportado en la sección anterior se incorpora efectivamente a la aplicación Android, transformando cada fotograma capturado por el `FrameSampler` en una lista de detecciones que la interfaz de usuario pueda mostrar.

5.4.1. Adaptación del *pipeline* de inferencia

La primera decisión de diseño fue cómo cargar y ejecutar el modelo `.tflite` desde Kotlin. Google ofrece dos vías principales: la librería de alto nivel `tflite-task-vision`, orientada a modelos con formatos estándar de entrada/salida, y el `Interpreter` de LiteRT, que permite el control directo sobre los tensores.

Se optó por la segunda vía, más manual pero necesaria en este caso: como se documentó en la Sección 5.3.6, el modelo exportado por Ultralytics espera un tensor de entrada en formato CHW y no en el HWC, que la librería de alto nivel asume por defecto. Emplear `tflite-task-vision` obligaría a intercambiar dimensiones antes de invocarla, complicando el código sin aportar ventaja.

La implementación se encapsula en la clase `GoProDetector`, que carga el modelo en el constructor con `FileUtil.loadMappedFile()` (mapea el fichero en memoria sin cargar todos los bytes) y configura el `Interpreter` con dos hilos y el delegado XNNPACK activado, que acelera la inferencia en CPU. Cada inferencia sigue el mismo flujo: se preprocesa el `Bitmap` a tensor CHW, se llama a `interpreter.run()`, se decodifica el tensor de salida y se aplica *Non-Maximum Suppression*.

El preprocesado es la parte más costosa del *pipeline*. Para cada fotograma se redimensiona el `Bitmap` a 640×640 con `Canvas.drawBitmap()`, reutilizando siempre el mismo `Bitmap` para no reservar memoria en cada iteración; se extraen los píxeles con `getPixels()` y se escriben en un `ByteBuffer` pre-asignado en orden CHW: primero los 409 600 valores del canal rojo normalizados a $[0, 1]$, luego el verde y por último el azul. Reutilizar así los *buffers* evita sobrecargar el recolector de basura de Android, algo clave para mantener una latencia estable entre inferencias.

5.4.2. Postprocesado: decodificación y *Non-Maximum Suppression*

El tensor de salida, de dimensiones `[1, 6, 8400]`, contiene 8400 detecciones candidatas. Para cada una, los seis valores son las coordenadas del centro y el tamaño del *bounding box* (cx, cy, w, h) más los *scores* de las dos clases. La inmensa mayoría de esas 8400 candidatas son ruido con *scores* muy bajos, y muchas de las válidas se solapan entre sí sobre el mismo objeto.

El postprocesado aplica dos filtros seguidos. Primero, un filtro de confianza: se descartan las candidatas cuyo *score* máximo entre las dos clases no llega a `CONFIDENCE_THRESHOLD` (0.25 por defecto), valor ajustado a mano buscando el equilibrio entre sensibilidad y falsos positivos. Después, un *Non-Maximum Suppression* (NMS) implementado a mano en Kotlin: se ordenan las detecciones que quedan por *score* descendente y, recorriéndolas de mayor a menor, se conserva cada una y se descartan las siguientes que se solapen con ella por encima de `IOU_THRESHOLD` (0.5). El resultado es una lista limpia de detecciones únicas, normalmente entre 0 y 5 por fotograma.

Cabe mencionar que Ultralytics ofrece una versión del exportador `.tflite` con el NMS ya integrado en el modelo. Se optó por implementarlo a mano en Kotlin por dos motivos: la versión integrada pesa más y da problemas de compatibilidad con ciertas versiones de LiteRT, y hacerlo a mano permite ajustar los umbrales sin tener que reexportar el modelo.

5.4.3. Muestreo periódico y control de *backpressure*

Aunque el *stream* de vídeo llega a la aplicación a 30 Hz, la inferencia no se ejecuta a esa frecuencia. El coste computacional en el dispositivo móvil hace que 30 inferencias por segundo sobrepasen la capacidad del procesador. Además, dado que el objetivo del sistema es alertar al piloto de tráficos que se mantienen visibles durante varios segundos, muestrear a mayor frecuencia no aporta información nueva. Se fijó una frecuencia de muestreo de **2 Hz** (una inferencia cada 500 ms).

El componente `FrameSampler` implementa este muestreo mediante un `Handler` sobre el hilo principal que ejecuta un `Runnable` cada 500 ms. Cada ejecución obtiene el `Bitmap` actual mediante `TextureView.getBitmap()` sobre un `Bitmap` reutilizado, y lo entrega al detector mediante una *callback*. Un mecanismo de control de *backpressure* basado en un `AtomicBoolean` evita solapar inferencias: si en el momento de disparar un muestreo

la inferencia anterior aún está en curso, el nuevo *frame* se descarta. Esto garantiza un consumo de recursos adecuado.

La Figura 5.10 resume el flujo completo de procesamiento de cada fotograma en la aplicación, desde su captura hasta la lista final de detecciones que se entrega a la interfaz.

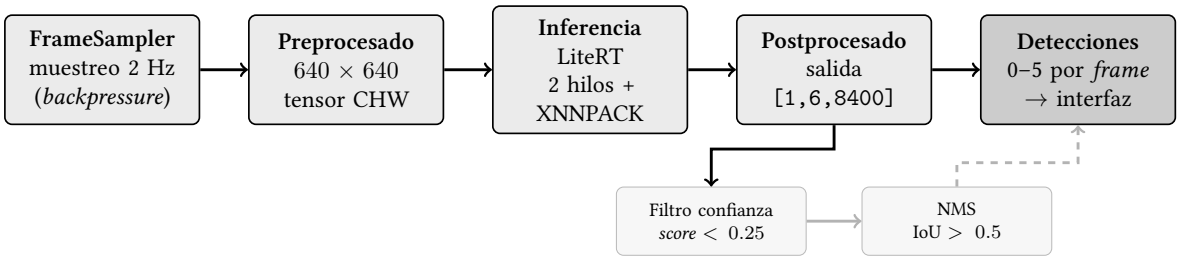


Figura 5.10: Flujo de procesamiento de cada fotograma: muestreo a 2 Hz con control de *backpressure*, preprocesado a tensor CHW, inferencia con LiteRT y postprocesado en dos etapas (filtro de confianza y NMS).

5.5. Interfaz de usuario

La interfaz de usuario de la aplicación se ha construido íntegramente con *Jetpack Compose* y *Material Design 3*. Es el componente que percibe finalmente el usuario y, por tanto, el que determina la utilidad práctica del sistema. El diseño pretende guiar al usuario a través del flujo secuencial de conexión con la cámara (BLE → WiFi → *stream*) sin exigir conocimientos técnicos previos y presentar las detecciones de forma clara sobre el vídeo en tiempo real.

5.5.1. Diseño visual: tema "Aviación"

Se ha definido una paleta de colores propia coherente con el dominio de la aplicación, inspirada en los paneles de instrumentos de aviación: fondos en azul oscuro profundo (`#0A1929`), superficies elevadas en tonos azul marino y acento principal en naranja radar (`#FF8C42`). Los estados del sistema utilizan un código de color estándar en aviónica [19]: verde para elementos activos y correctamente conectados, ámbar para avisos y rojo para errores.

El tema se aplica siempre en modo oscuro, con independencia de la configuración del sistema. Mejora la legibilidad del vídeo capturado al dar suficiente contraste y reduce la

luz que emite la pantalla en condiciones de cabina. La Figura 5.11 muestra la aplicación con el tema descrito.

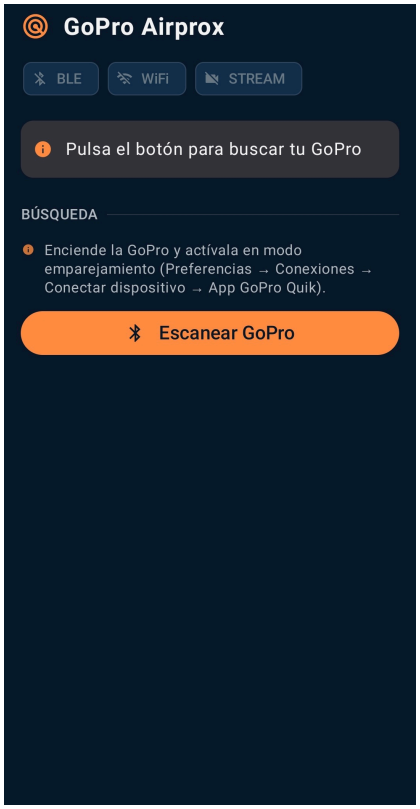


Figura 5.11: Aspecto general de la aplicación con el tema "Aviación" aplicado: fondo azul profundo, acento naranja, tipografía técnica.

5.5.2. Flujo progresivo de conexión

La pantalla principal se estructura en torno a una cabecera compacta con el nombre de la aplicación y tres *chips* de estado (BLE, WiFi, *stream*) que cambian de color en función del estado de cada canal. Debajo, un área principal muestra un botón grande para escanear cámaras GoPro cercanas, y una vez descubiertas, una tarjeta (`DeviceCard`) por cada una.

En lugar de mostrar todas las acciones disponibles simultáneamente, la tarjeta va revelando secciones a medida que el estado del flujo lo justifica. Al inicio solo se muestra la sección de conexión BLE. Al establecerse la conexión, aparecen los controles de la cámara (grabar, activar el punto de acceso WiFi, leer credenciales). Una vez leídas las credenciales, se muestra la sección WiFi con SSID y contraseña más el botón de conexión. Cuando el

dispositivo se une a la red de la cámara, aparece finalmente la sección de *stream* con los botones de inicio y parada.

Este enfoque reduce la carga cognitiva del usuario, que solo ve las acciones relevantes; y guía el flujo correcto, evitando que el usuario intente ejecutar acciones fuera de orden. Se complementa con textos de ayuda (`HintText`) que aparecen en los puntos clave del flujo. La Figura 5.12 ilustra la evolución de la tarjeta `DeviceCard` conforme avanza el flujo de conexión.

5.5.3. Superposición de detecciones sobre el vídeo

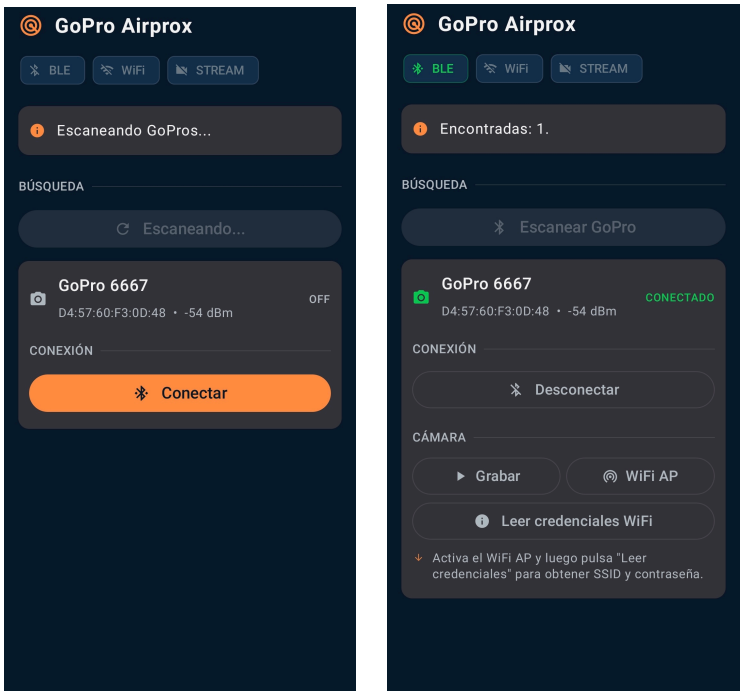
Durante el *stream*, la interfaz combina en la misma región de la pantalla tres capas visuales apiladas: el vídeo (`VLCVideoLayout` de LibVLC dentro de un `AndroidView`), un *overlay* de detecciones dibujado con `Canvas` de Compose, y un *Head-Up Display* (HUD)[8] informativo. La Figura 5.13 muestra el resultado final con las tres capas.

El *overlay* de detecciones dibuja para cada *bounding box* detectado cuatro esquinas remarcadas al estilo de las miras de aviónica (*target scope*), acompañadas de un relleno semitransparente. Este estilo se ha elegido porque resulta menos invasivo sobre el vídeo. El grosor del trazo se hace variable en función del *score* de la detección (más grueso a mayor confianza) y el color varía según un cálculo de nivel de amenaza que combina *score* y tamaño del *bounding box*: rojo para amenaza alta, naranja para media y amarillo para baja o para detecciones de la clase `bird`. Cada detección lleva sobre su *bounding box* una etiqueta con formato `#N AIRCRAFT XX%`, donde `N` es un identificador de seguimiento (véase Sección 5.5.4), la clase se escribe en mayúsculas y el porcentaje corresponde al *score*.

El HUD se sitúa en la parte superior del vídeo. Muestra el número total de detecciones activas, el *score* máximo de todas ellas y latencia de la última inferencia en milisegundos. Este último dato es útil durante las pruebas descritas en el Capítulo 6 porque permite verificar en tiempo real la latencia del sistema. Se añade también una cruz (*crosshair*) en el centro de la imagen y un pequeño panel en la esquina inferior derecha con la referencia del modelo y del *dataset* empleados.

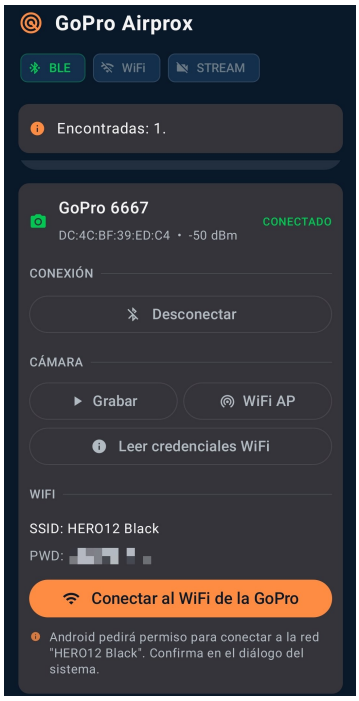
Debajo del vídeo, una banda horizontal de tarjetas (`LazyRow`) muestra el historial de

[8]Panel informativo superpuesto sobre la imagen, habitual en aviónica, que muestra datos de estado sin obstruir la visión del contenido principal.

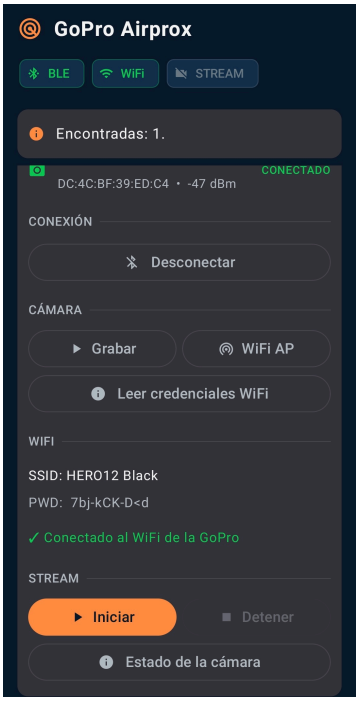


(a) BLE

(b) Cámara



(c) WiFi



(d) Stream

Figura 5.12: Evolución de la tarjeta `DeviceCard` durante el flujo de conexión. Las secciones aparecen a medida que el estado del sistema las hace relevantes: (a) tras el escaneo BLE, (b) tras la conexión BLE, (c) tras leer las credenciales WiFi, (d) tras conectarse al punto de acceso de la cámara.



Figura 5.13: Ejemplo de superposición del *overlay* de detecciones sobre el vídeo capturado por la GoPro.

las últimas detecciones con su clase, *score* y tiempo transcurrido. Este componente permite documentar el historial. La Figura 5.14 muestra la banda de historial en operación.

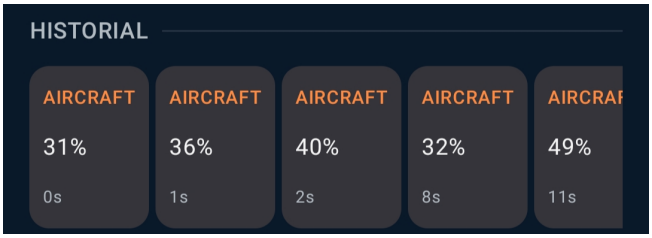


Figura 5.14: Banda horizontal con las últimas detecciones registradas.

5.5.4. Seguimiento simple entre fotogramas

Un problema visual asociado a mostrar detecciones sobre vídeo en tiempo real es la falta de identidad persistente: cada fotograma se procesa independientemente y las detecciones "parpadean" con identificadores distintos aunque correspondan al mismo objeto. Para evitarlo se ha implementado un *tracking* simple por proximidad entre fotogramas: al recibir una nueva lista de detecciones, cada una se intenta emparejar con una del fotograma anterior de la misma clase cuya diferencia en posición del centro sea menor de un umbral (0.1 en coordenadas normalizadas); si se encuentra pareja, hereda el identificador de la anterior, si no, se le asigna uno nuevo. Este mecanismo mantiene la coherencia de los identificadores mientras un tráfico aéreo permanece dentro del campo de visión.

6. Resultados

En este capítulo se recogen los resultados de la validación experimental del sistema desarrollado. Se estructura en tres bloques: rendimiento del modelo de detección sobre el conjunto de validación (Sección 6.1), caracterización del funcionamiento del sistema en los dispositivos móviles empleados (Sección 6.2), y validación empírica del *pipeline* completo integrado con la cámara real (Sección 6.3). El objetivo no es reproducir las métricas ya presentadas en el Capítulo 5, sino cerrar el ciclo de evaluación del sistema como un todo funcional.

6.1. Rendimiento del modelo de detección

Como se detalló en la Sección 5.3.5, el modelo YOLOv8n entrenado alcanza sobre el conjunto de validación (1197 imágenes) una *mAP@0.5* de 0.993 en la clase `aircraft` (P = 0.967, R = 0.982). La Tabla 6.1 muestra estos valores frente a una referencia mínima aceptada.

Métrica	**Umbral mínimo**	**Modelo TFM (`aircraft`)**	**Cumplimiento**
mAP@0.5	≥ 0.60	0.993	✓
Precision	—	0.967	—
Recall	≥ 0.85	0.982	✓

Tabla 6.1: Comparación de las métricas obtenidas por el modelo sobre el conjunto de validación (1197 imágenes, resolución 1280 px) con los umbrales mínimos fijados en este trabajo. La latencia de inferencia en el dispositivo móvil se caracteriza en la Sección 6.2.

Un análisis complementario del comportamiento del modelo sobre los distintos escenarios del *dataset* (*above horizon* versus *below horizon*) muestra que la mayor parte de errores

de *recall* sobre `aircraft` se concentran en las secuencias *below horizon* con fondos de *clutter* terrestre, lo cual es coherente con las dificultades documentadas en la literatura para este subproblema [12]. La clase `bird`, con un *recall* del 47 %, es la que muestra el mayor margen de mejora; su comportamiento es aceptable en el contexto de la aplicación pero se documenta como línea de trabajo futuro.

6.2. Caracterización en el dispositivo móvil

Aunque las métricas del modelo se obtuvieron sobre GPU en la nube durante la validación, el objetivo del TFM es que el sistema funcione sobre *smartphone*. La Tabla 6.2 recoge la latencia de inferencia real medida directamente en la aplicación en dos dispositivos con arquitecturas distintas.

Dispositivo	**SoC**	**Latencia inferencia**
NVIDIA L4 (referencia)	Ada Lovelace	3.5 ms
Xiaomi Redmi Note 10	Snapdragon 678	60–90 ms
Xiaomi Redmi Note 8	MediaTek Helio G85	80–130 ms

Tabla 6.2: Latencia de la inferencia LiteRT (delegado XNNPACK) en cada dispositivo. Se incluye la GPU L4 como referencia. El valor que muestra el HUD de la aplicación es distinto: corresponde al tiempo total por fotograma (preprocesado, inferencia y postprocesado), que ronda el segundo.

Estos valores permiten dos observaciones. Primero, aunque la latencia de inferencia en el móvil es aproximadamente 20× superior a la de la GPU de referencia, no es el cuello de botella del sistema: el tiempo total por fotograma, es de un segundo, de modo que el control de *backpressure* descrito en la Sección 5.4.3 descarta parte de los muestreos y la frecuencia efectiva de detección queda en torno a 1 Hz.. Segundo, la diferencia entre el Snapdragon 678 y el MediaTek Helio G85 es notable (aproximadamente 40 %) y es coherente con la diferencia de generación y de capacidad de sus procesadores.

Cabe destacar que la limitación operativa del Redmi Note 8 documentada en la Sección 5.2.3 *no* corresponde a la latencia del modelo (que aquí se muestra aceptable) sino al *pipeline* de captura de fotogramas: el *cuello de botella* en ese dispositivo está en la interacción entre LibVLC y el compositor gráfico de HyperOS, no en la inferencia LiteRT. La Figura 6.1 muestra el HUD de la aplicación durante una prueba real, con la latencia mostra-

da en tiempo real.

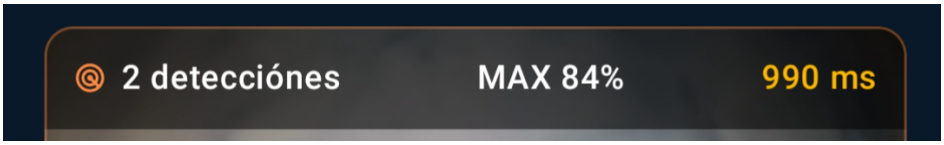


Figura 6.1: HUD superior de la aplicación durante operación real, mostrando el número de detecciones activas, el *score* máximo y la latencia de la última inferencia.

6.3. Validación empírica con GoPro real

La validación final del sistema se ha realizado con la cámara GoPro Hero 12 real, integrada con el dispositivo Xiaomi Redmi Note 10 mediante el flujo completo BLE → WiFi → HTTP → *stream*, y con el modelo entrenado ejecutándose sobre los fotogramas capturados. Al no disponer de acceso operativo a una aeronave en vuelo, se han diseñado dos protocolos de prueba complementarios que evalúan aspectos distintos del sistema.

6.3.1. Protocolo 1: proyección de imágenes del propio *dataset*

En este protocolo se han seleccionado 20 imágenes representativas del subconjunto AOT (no incluidas en el conjunto de *val* del entrenamiento) y se han mostrado a pantalla completa en la pantalla de un ordenador. La GoPro se ha situado a aproximadamente 40–50 cm de la pantalla, ajustando el encuadre para que la imagen ocupara la mayor parte del campo de visión de la cámara, y se ha registrado si la aeronave presente en cada imagen resultaba detectada por el sistema durante los 10 segundos siguientes al arranque del *stream*.

Los resultados de este protocolo se resumen en la Tabla 6.3. Este protocolo es representativo del escenario ideal para el modelo (imágenes con distribución exactamente igual a la de entrenamiento), y por tanto su tasa de éxito debe interpretarse como una *cota superior* realista del rendimiento del sistema en condiciones de campo.

Escenario del *dataset*	Imágenes probadas	Detecciones exitosas	Tasa
Above horizon (fondo cielo)	10	9	90 %
Below horizon (fondo terreno)	10	7	70 %
Total	20	16	80 %

Tabla 6.3: Resultados del protocolo 1. Se considera detección exitosa la aparición de al menos un *bounding box* sobre la aeronave real durante la ventana de 10 segundos.

Cabe señalar que los *scores* de las detecciones exitosas en este protocolo se sitúan típicamente entre el 50 % y el 75 %, muy por debajo del 99 % reportado en las métricas de validación. Esta diferencia se atribuye a la degradación del contenido visual introducida por la cadena de proyección: la imagen original en escala de grises se muestra en una pantalla con retroiluminación, se captura por la GoPro con su óptica y sensor, se codifica en H.264, se transmite por UDP, se decodifica en el móvil y finalmente se redimensiona a 640×640. Cada uno de estos pasos introduce artefactos que se acumulan.

6.3.2. Protocolo 2: escenarios reales al aire libre

El segundo protocolo consiste en apuntar la GoPro al cielo desde una localización con tráfico aéreo visible ocasional (líneas de vuelo comerciales, aeronaves ligeras). Este protocolo es cualitativo por su naturaleza no reproducible, no se controla qué aeronaves aparecen ni cuándo, pero es el más representativo del uso operativo del sistema.

Los resultados observados durante las sesiones de prueba muestran que el sistema detecta aeronaves comerciales a distancias medias (estimación entre 2 y 5 km, con la aeronave ocupando entre 15 y 40 píxeles del *frame*) con *scores* entre 30 % y 60 %. Las detecciones son intermitentes: un mismo objeto puede detectarse en algunos fotogramas y no en los siguientes, comportamiento que el mecanismo de *tracking* simple descrito en la Sección 5.5.4 mitiga parcialmente al mantener la coherencia visual del identificador. La Figura 6.2 muestra un ejemplo real de detección de aeronave comercial durante una de estas pruebas.

Estos resultados confirman la viabilidad del sistema propuesto en condiciones representativas del uso real, con las siguientes limitaciones identificadas: la degradación de *score* respecto a las métricas de validación es significativa (del 99 % al rango 30–60 %),

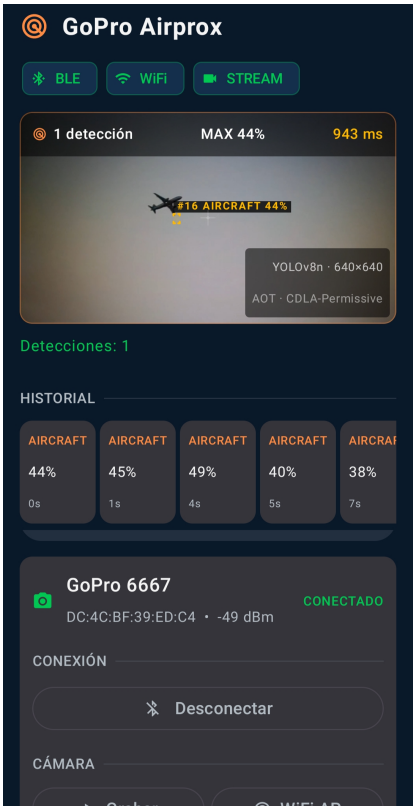


Figura 6.2: Ejemplo de detección real durante una prueba al aire libre en las inmediaciones del Aeropuerto de Valencia. La aeronave, un avión comercial en fase de crucero visible como un pequeño objeto sobre fondo de cielo, es detectada por el sistema con *score* moderado.

los objetos deben permanecer al menos algunos fotogramas dentro del campo de visión para que las detecciones intermitentes resulten útiles al piloto, y las condiciones adversas de iluminación (contraluz, cielo con nubes con alto contraste) no se han caracterizado sistemáticamente en este trabajo.

Todas estas limitaciones son consistentes con las expectativas académicas para un sistema entrenado con un subconjunto reducido del *dataset* y sin ajustes específicos de campo. La mejora de la robustez a condiciones reales queda como línea de trabajo futuro, como se discute en el Capítulo 7.

7. Conclusiones y trabajo futuro

Este capítulo cierra la memoria haciendo balance del trabajo realizado. Se valora en primer lugar el grado de consecución de los objetivos planteados al inicio del proyecto. A continuación se extraen las conclusiones generales que el desarrollo ha permitido alcanzar. Se justifican después las competencias del Máster que la elaboración del TFM ha permitido consolidar. Por último, se proponen líneas de trabajo futuro que dan continuidad a lo desarrollado, tanto en el ámbito del propio sistema como en su evolución hacia un producto operativo en el marco de la aviación.

7.1. Cumplimiento de los objetivos

Se valora a continuación el grado de consecución de los objetivos planteados en el Capítulo 2.

- **Objetivo 1 — Comunicación entre el dispositivo móvil y la cámara**: cumplido. La aplicación establece la conexión BLE con la GoPro Hero 12, activa programáticamente su punto de acceso WiFi, lee las credenciales, se une a la red y controla las funciones relevantes de la cámara mediante llamadas HTTP a la interfaz Open GoPro.
- **Objetivo 2 — Creación y entrenamiento del modelo**: cumplido. El modelo YOLOv8n entrenado sobre el subconjunto AOT alcanza, sobre el conjunto de validación, $mAP@0.5 = 0.993$ y un *recall* de 0.982 en la clase `aircraft`, por encima de los umbrales mínimos fijados (≥ 0.60 y ≥ 0.85, respectivamente).
- **Objetivo 3 — Integración del modelo en la aplicación Android**: cumplido. El modelo se ejecuta *on-device* mediante LiteRT sobre los fotogramas capturados del *stream*, con latencias de inferencia entre 60 ms y 130 ms según dispositivo, muy por debajo del intervalo de muestreo configurado.

- **Objetivo 4 — Diseño de la interfaz de usuario**: cumplido. La interfaz superpone las detecciones sobre el vídeo con *bounding boxes* tipo *target scope*, un HUD con métricas del detector, un mecanismo simple de *tracking* entre fotogramas para mantener la coherencia visual y un flujo progresivo de conexión que guía al usuario sin conocimientos técnicos previos.
- **Objetivo 5 — Experimentación y validación del sistema**: cumplido. La validación combina métricas objetivas sobre 1197 imágenes independientes del entrenamiento y pruebas empíricas del *pipeline* completo con la cámara real, en dos protocolos complementarios descritos en el Capítulo 6.

Por tanto, se considera alcanzado el objetivo general del TFM: se dispone de un sistema funcional de "ojo artificial" complementario para la detección de amenazas aéreas asociadas al tráfico no cooperativo, viable como aplicación de apoyo a la decisión del piloto sobre *smartphone* estándar.

7.2. Conclusiones generales

Más allá de los objetivos planteados, el desarrollo del proyecto ha permitido extraer algunas conclusiones generales que se recogen a continuación.

En primer lugar, la **viabilidad de la detección visual de tráfico aéreo en dispositivos móviles de gama baja es real**. La combinación de un modelo compacto (YOLOv8n) con un motor de inferencia moderno (LiteRT) sobre CPU con delegado XNNPACK proporciona tiempos de inferencia de decenas de milisegundos, sin necesidad de aceleradores especializados.

En segundo lugar, la **arquitectura del *stream* de vídeo es el punto más frágil del sistema** cuando se implementa sobre hardware heterogéneo. La incompatibilidad detectada entre el compositor gráfico de HyperOS sobre chipsets MediaTek de gama de entrada y la ruta de rendering de LibVLC refleja que las decisiones de diseño en este bloque tienen implicaciones prácticas. Este hallazgo refleja la fragmentación de Android.

En tercer lugar, la **diversidad de escenarios en los datos de entrenamiento es más determinante que su volumen absoluto**. La primera iteración del entrenamiento, con división *train/val* por secuencia sobre solo cinco secuencias del AOT, produjo un modelo

que no generalizaba en absoluto. El resultado confirma un principio conocido en *Machine Learning*: para generalizar hace falta diversidad, no abundancia.

Por último, el proyecto ha reforzado la idea de que **documentar los descartes técnicos aporta valor**. Las tres alternativas evaluadas para el *stream* (Media3, `MediaCodec` propio, LibVLC) y las dos estrategias probadas de división de datos aparecen como parte del aprendizaje del TFM y como referencia para quien pueda continuar este trabajo.

7.3. Competencias adquiridas

La elaboración de este TFM ha permitido demostrar y consolidar las siguientes competencias del Máster Universitario en Inteligencia Artificial:

- **CN02 — Comprender los fundamentos teóricos y prácticos de los métodos y herramientas de Inteligencia Artificial para solucionar problemas complejos**. La elección justificada de YOLOv8n frente a otras variantes, el manejo del formato de tensores CHW (Sección 5.3.6) y la implementación del algoritmo de *Non-Maximum Suppression* (Sección 5.4.2) evidencian el dominio de los fundamentos de los detectores basados en redes neuronales convolucionales.
- **HA01 — Desarrollar sistemas software que usen técnicas de aprendizaje automático generando aplicaciones prácticas**. El TFM culmina en una aplicación Android funcional que ejecuta inferencia de un modelo de aprendizaje automático en tiempo real sobre un dispositivo móvil, integrando el modelo con el resto de componentes del sistema.
- **HA02 — Evaluar qué técnicas y métodos avanzados son los más adecuados para la resolución de problemas que requieren el uso de métodos de la Inteligencia Artificial**. La comparación entre estrategias de división del *dataset* (por secuencia frente a por fotograma, Sección 5.3.3), la elección de *transfer learning* frente a entrenamiento desde cero, y la selección de LiteRT frente a otros *frameworks* de inferencia embarcada, muestran una evaluación de alternativas.
- **CP01 — Aplicar conceptos, técnicas y métodos avanzados de Inteligencia Artificial en entornos que requieran percepción e interacción con el entorno**. El sistema desarrollado constituye un caso de percepción por visión por computador aplicada a un entorno físico real con interacción con el usuario final (piloto) mediante una interfaz específicamente diseñada.

- **CP02 — Planificar y ejecutar proyectos de desarrollo y despliegue de soluciones a problemas complejos conforme a procesos de ingeniería de proyectos**. El proyecto se ha estructurado en subhitos con validación individual (Capítulo 4), se ha planificado temporalmente (Figura 4.1) y se ha documentado siguiendo criterios de ingeniería.

7.4. Líneas de trabajo futuro

A partir del sistema desarrollado se identifican varias líneas de mejora que constituyen continuaciones naturales del trabajo.

7.4.1. Ampliación del conjunto de entrenamiento

El modelo se ha entrenado con un subconjunto de cinco secuencias del AOT (aproximadamente 6000 imágenes). Ampliar este subconjunto cubriría una mayor diversidad de condiciones de iluminación y tipos de aeronave, con una mejora esperable del *recall*. La infraestructura de entrenamiento y el *pipeline* de preparación del *dataset* son reutilizables.

7.4.2. Distinción de escenarios airprox

El sistema actual detecta la presencia de aeronaves, pero no distingue el tipo de escenario de aproximación (*close-in*, *head-to-head*, *crossing*). Un análisis de la evolución del *bounding box* entre fotogramas, crecimiento del tamaño como indicador de aproximación frontal, permitiría clasificar el tipo de escenario.

7.4.3. Modo operativo minimalista

La interfaz actual muestra información técnicamente rica que ha resultado útil durante el desarrollo y la validación. Para el uso operativo real en cabina sería recomendable ofrecer un modo alternativo minimalista, minimizando la distracción del piloto. La estructura modular del sistema facilita esta extensión.

7.4.4. Certificación y despliegue operativo

El sistema es una prueba de concepto: cualquier evolución hacia un producto operativo real quedaría fuera del alcance de un TFM y requeriría caracterización sistemática de falsos positivos y negativos en vuelo controlado, integración con estándares como ASTM F3442 [10], y cumplimiento de los requisitos de las agencias correspondientes (EASA, FAA).

Referencia bibliográfica

[1] FLARM Technology. Flarm – flight alarm. https://www.flarm.com/en/, 2024.

[2] EASA. Avoiding mid-air collisions. https://www.easa.europa.eu/community/topics/sunny-swift-41-avoiding-mid-air-collisions, 2023.

[3] Agencia Estatal de Seguridad Aérea. Promoción de la seguridad aérea. https://www.seguridadaerea.gob.es/es/prom-de-seguridad, 2024.

[4] Real Aeroclub de España. Aerial guard: IA en la detección y rastreo de objetos voladores. https://xn--realaeroclubdeespaa-d4b.org/aerial-guard-ia-en-la-deteccion-y-rastreo-de-objetos-voladores/, 2023.

[5] Aerial Guard. Demostración de detección de tráfico aéreo no cooperativo. https://www.youtube.com/watch?v=UWHgGmfqCak, 2021. Vídeo.

[6] GoPro. Open GoPro. https://gopro.com/en/us/info/open-gopro, 2024.

[7] GoPro. Open GoPro – compatibility. https://gopro.github.io/OpenGoPro/#compatibility, 2024.

[8] Google. LiteRT (antes TensorFlow Lite). https://ai.google.dev/edge/litert?hl=es-419, 2024.

[9] Amazon Prime Air and AIcrowd. Airborne object tracking challenge. https://www.aicrowd.com/challenges/airborne-object-tracking-challenge, 2021.

[10] Aaron McFadyen and Luis Mejias. A survey of autonomous vision-based See and Avoid for unmanned aircraft systems. *Progress in Aerospace Sciences*, 80:1–17, 2016.

[11] John Lai, Jason J. Ford, Luis Mejias, and Peter O'Shea. Characterization of sky-region morphological-temporal airborne collision detection. *Journal of Field Robotics*, 30(2):171–193, 2013.

[12] Jasmin James, Jason J. Ford, and Timothy L. Molloy. Below horizon aircraft detection using deep learning for vision-based sense and avoid. *arXiv preprint arXiv:1903.03275*, 2019. https://arxiv.org/abs/1903.03275.

[13] Fatih Cagatay Akyon, Sinan Onur Altinuc, and Alptekin Temizel. Slicing aided hyper inference and fine-tuning for small object detection. In *2022 IEEE International Conference on Image Processing (ICIP)*, pages 966–970, 2022.

[14] Sourish Ghosh, Jay Patrikar, Brady Moon, Milad Hamidi, and Sebastian Scherer. AirTrack: Onboard deep learning framework for long-range aircraft detection and tracking. *arXiv preprint arXiv:2209.12849*, 2022. https://arxiv.org/abs/2209.12849.

[15] Elysia Q. Smith, Amrutha Vairavanathan, et al. AVOIDDS: Aircraft vision-based intruder detection dataset and simulator. *arXiv preprint arXiv:2306.11203*, 2023. https://arxiv.org/abs/2306.11203.

[16] Iris Automation. Casia: Detect-and-avoid. https://www.irisonboard.com/casia/, 2024.

[17] VideoLAN. libvlc: multimedia framework. https://www.videolan.org/vlc/libvlc.html, 2024. Consultado en agosto de 2026.

[18] Ultralytics. Yolov8. https://github.com/ultralytics/ultralytics, 2023. Consultado en agosto de 2026.

[19] Federal Aviation Administration. Advisory Circular AC 25.1322-1: Flight Crew Alerting. Advisory Circular AC 25.1322-1, U.S. Department of Transportation, FAA, 2010.

ANEXOS

A. Código fuente relevante

Este anexo recoge los fragmentos de código considerados más significativos del proyecto. Se ha priorizado la inclusión de los componentes que contienen decisiones técnicas relevantes documentadas en el Capítulo 5, evitando el código auto-explicativo o de *plumbing* habitual en aplicaciones Android. La totalidad del código fuente está disponible en el repositorio Git asociado al proyecto, cuya ubicación se detalla en el Anexo B.

A.1. Preprocesado CHW y ejecución del modelo

La clase `GoProDetector` encapsula la carga del modelo LiteRT, el preprocesado de `Bitmap` al formato de tensor CHW requerido, la ejecución del `Interpreter`, y el postprocesado con *Non-Maximum Suppression*. El Código A.1 reproduce el método de preprocesado, que es donde reside la particularidad del formato CHW frente al HWC habitual.

El Código A.2 reproduce la implementación de *Non-Maximum Suppression* aplicada al tensor de salida del modelo tras el filtrado por umbral de confianza.

A.2. Muestreo periódico de fotogramas

La clase `FrameSampler` implementa el muestreo periódico de fotogramas mediante `TextureView.getBitmap()`, como se recoge en el Código A.3. Incluye control de *backpressure* y detección de fallos consecutivos para reintentar la conexión con la vista si el mecanismo deja de responder.

Código A.1: GoProDetector: preprocesado de `Bitmap` a tensor CHW

```
private fun preprocess(source: Bitmap) {
    // Escalar el bitmap a 640x640 sobre buffer reutilizado.
    val canvas = Canvas(scaledBitmap)
    val srcRect = Rect(0, 0, source.width, source.height)
    val dstRect = Rect(0, 0, INPUT_SIZE, INPUT_SIZE)
    canvas.drawBitmap(source, srcRect, dstRect, null)

    val pixels = IntArray(INPUT_SIZE * INPUT_SIZE)
    scaledBitmap.getPixels(pixels, 0, INPUT_SIZE, 0, 0, INPUT_SIZE, INPUT_SIZE)

    inputBuffer.rewind()

    // Formato CHW: primero R, luego G, luego B, cada canal contiguo en memoria.
    for (p in pixels) {
        inputBuffer.putFloat(((p shr 16) and 0xFF) / 255f)
    }
    for (p in pixels) {
        inputBuffer.putFloat(((p shr 8) and 0xFF) / 255f)
    }
    for (p in pixels) {
        inputBuffer.putFloat((p and 0xFF) / 255f)
    }
}
```

Código A.2: GoProDetector: implementación manual de NMS

```
private fun nonMaxSuppression(detections: List<Detection>): List<Detection> {
    val sorted = detections.sortedByDescending { it.score }.toMutableList()
    val kept = mutableListOf<Detection>()

    while (sorted.isNotEmpty()) {
        val best = sorted.removeAt(0)
        kept.add(best)
        // Descartar candidatas de la misma clase con IoU > umbral.
        sorted.removeAll { candidate ->
            candidate.classId == best.classId && iou(best, candidate) > IOU_THRESHOLD
        }
    }
    return kept
}

private fun iou(a: Detection, b: Detection): Float {
    val interLeft = maxOf(a.boxLeft, b.boxLeft)
    val interTop = maxOf(a.boxTop, b.boxTop)
    val interRight = minOf(a.boxRight, b.boxRight)
    val interBottom = minOf(a.boxBottom, b.boxBottom)
    val interW = maxOf(0f, interRight - interLeft)
    val interH = maxOf(0f, interBottom - interTop)
    val interArea = interW * interH
    val areaA = a.boxW * a.boxH
    val areaB = b.boxW * b.boxH
    val union = areaA + areaB - interArea
    return if (union > 0f) interArea / union else 0f
}
```

Código A.3: FrameSampler: captura de fotograma sobre `TextureView`

```
private fun captureOnce() {
    val textureView = findTextureView(videoLayout)
    if (textureView == null || !textureView.isAvailable) {
        AirproxLog.v(TAG, "TextureView aun no disponible.")
        return
    }
    try {
        textureView.getBitmap(bitmap)
        consecutiveFailures = 0
        onFrame?.invoke(bitmap)
    } catch (e: Exception) {
        consecutiveFailures++
        AirproxLog.w(TAG, "Error en getBitmap: ${e.message} (fallos: $consecutiveFailures
    )")
        if (consecutiveFailures >= MAX_CONSECUTIVE_FAILURES) {
            AirproxLog.w(TAG, "Demasiados fallos consecutivos, reset necesario")
            onResetNeeded?.invoke()
            consecutiveFailures = 0
        }
    }
}
```

A.3. Conversión del *dataset* AOT al formato YOLO

El Código A.4 (Python, en *notebook* Jupyter) resume el proceso de conversión de las anotaciones del AOT al formato normalizado esperado por Ultralytics YOLOv8. Se muestra únicamente el núcleo del proceso: filtrado por secuencias seleccionadas, mapeo de categorías a clases YOLO y normalización de las coordenadas.

Código A.4: Conversión de anotaciones AOT al formato YOLO

```
# Filtrar solo las 5 secuencias seleccionadas
df_subset = df[df["flight_id"].isin(FLIGHT_IDS)].copy()

# Descartar filas sin bbox o sin id (14 filas anomalas)
df_clean = df_subset[df_subset["id"].notna()].copy()
df_clean = df_clean[df_clean["gt_left"].notna()].copy()

# Mapear categorias del dataset a clases YOLO
def id_to_class(obj_id):
    if obj_id in ["Airplane1", "Helicopter1"]:
        return 0  # aircraft (fusion)
    elif obj_id == "Bird1":
        return 1  # bird
    return None

df_clean["yolo_class"] = df_clean["id"].apply(id_to_class)

# Formato YOLO: class x_center y_center width height (todo normalizado [0,1])
df_clean["x_center"] = ((df_clean["gt_left"] + df_clean["gt_right"]) / 2) / IMG_WIDTH
df_clean["y_center"] = ((df_clean["gt_top"] + df_clean["gt_bottom"]) / 2) / IMG_HEIGHT
df_clean["width_norm"]  = (df_clean["gt_right"]  - df_clean["gt_left"]) / IMG_WIDTH
df_clean["height_norm"] = (df_clean["gt_bottom"] - df_clean["gt_top"])  / IMG_HEIGHT
```

A.4. Configuración de LibVLC y adjunción de vista

Los parámetros específicos de arranque de LibVLC, orientados a minimizar latencia y la forma de adjuntar la vista para forzar el uso de `TextureView` en lugar de `SurfaceView`, se recogen en el Código A.5.

Código A.5: GoProStreamPlayer: configuración de LibVLC

```
private val libVlc: LibVLC by lazy {
    val options = arrayListOf(
        "--network-caching=100",
        "--live-caching=100",
        "--file-caching=100",
        "--clock-jitter=0",
        "--clock-synchro=0",
        "-vvv",
    )
    LibVLC(context, options)
}

fun attachView(videoLayout: VLCVideoLayout) {
    // Ultimo parametro: true => TextureView (permite lectura por getBitmap).
    mediaPlayer.attachViews(videoLayout, null, false, true)
}

fun start() {
    val media = Media(libVlc, Uri.parse("udp://@:8554"))
    media.setHWDecoderEnabled(true, false)
    mediaPlayer.media = media
    media.release()
    mediaPlayer.play()
}
```

B. Guía de instalación y despliegue

Este anexo describe los pasos necesarios para instalar, compilar y ejecutar el sistema completo desarrollado en el presente Trabajo Fin de Máster. Se cubre tanto la aplicación Android (compilación e instalación en un dispositivo físico) como el pipeline de entrenamiento del modelo (por si se desea reentrenar o extender el conjunto de datos).

B.1. Repositorio del proyecto

El código fuente completo del proyecto se encuentra en el repositorio Git `https://github.com/OscarDominguezOcana/tfm`, accesible también mediante el código QR mostrado en la Figura B.1.



Figura B.1: Código QR con el enlace al repositorio Git del proyecto en GitHub.

El repositorio contiene la estructura habitual de un proyecto Android moderno organizado como *multi-module*: el módulo `app` contiene la actividad principal y la interfaz Compose, mientras que el módulo `gopro` encapsula toda la lógica de comunicación con la cámara (BLE, WiFi, HTTP), el reproductor de *stream* y el detector LiteRT.

B.2. Requisitos previos

Para compilar y ejecutar el sistema son necesarios los siguientes componentes:

- **Android Studio Ladybug** o superior, con soporte para Kotlin 2.x y AGP 8.10.x.
- **JDK 17** (Temurin recomendado). Android Studio incluye una versión embebida; para compilaciones desde línea de comandos, verificar que `JAVA_HOME` apunta a un JDK 17.
- **Android SDK**, con las plataformas API 30 (mínima) a API 35 (compilación).
- **Un dispositivo Android físico** con:
  - Android 11 (API 30) o superior.
  - Bluetooth Low Energy.
  - Compatibilidad hardware con `TextureView` + LibVLC (validado en Xiaomi Redmi Note 10, Snapdragon 678).
- **Una cámara GoPro compatible con Open GoPro**, validada con la Hero 12 Black. El resto de la familia Hero 10 y posteriores debería funcionar sin modificaciones.

Adicionalmente, para reentrenar el modelo:

- **Cuenta de Google** con acceso a **Google Colab Pro** (recomendado para sesiones de entrenamiento sin interrupciones) y **Google Drive** con al menos 20 GB libres.
- **Python 3.10** o superior en entorno local con las librerías `pandas`, `Pillow`, `matplotlib`, `jupyter`.
- **AWS CLI** configurada para descargas anónimas (`--no-sign-request`).

B.3. Compilación e instalación de la aplicación

1. Clonar el repositorio:
   `git clone` *`URL_repositorio`* `&& cd` *`directorio`*
2. Abrir el proyecto en Android Studio (File → Open, seleccionar la carpeta raíz).
3. Esperar a que Gradle sincronice todas las dependencias.
4. Conectar el dispositivo Android por USB con *depuración USB* habilitada.
5. En Xiaomi/MIUI, habilitar también *instalar aplicaciones vía USB*.

6. Autorizar el emparejamiento del ordenador cuando el dispositivo lo solicite.
7. Pulsar `Run` (Shift+F10) para compilar e instalar.

La primera compilación puede tardar entre 2 y 5 minutos por la descarga inicial de dependencias. Compilaciones incrementales posteriores son del orden de segundos.

B.4. Configuración inicial de la cámara

Antes del primer uso de la aplicación con una cámara GoPro concreta, es necesario emparejarla previamente con el dispositivo Android mediante la aplicación oficial *GoPro Quik*[1]. El motivo es que el sistema Bluetooth de Android requiere un vínculo previo autorizado en la cámara para permitir la conexión GATT desde aplicaciones de terceros.

1. Instalar GoPro Quik desde Google Play.
2. Encender la cámara y ponerla en modo emparejamiento (Preferencias → Conexiones → Conectar dispositivo → App GoPro Quik).
3. Desde Quik, seguir el asistente para emparejar la cámara. Al finalizar, el emparejamiento queda registrado.
4. **Importante**: si Quik ofrece actualizar el *firmware* de la cámara, se recomienda declinar durante el uso de este TFM para preservar la compatibilidad validada.
5. Cerrar Quik. Ya se puede iniciar la aplicación desarrollada, que detectará la cámara sin problemas.

B.5. Uso de la aplicación

El flujo de uso normal es el siguiente:

1. Encender la cámara GoPro.
2. Abrir la aplicación en el dispositivo Android.
3. Conceder los permisos solicitados (Bluetooth, ubicación –requerida por Android para escaneos BLE–, dispositivos cercanos).
4. Pulsar `Escanear GoPro`. En unos segundos, la cámara aparecerá en la lista.

[1]Este paso solo es necesario la primera vez. Posteriores conexiones desde la aplicación desarrollada funcionarán directamente por el BLE registrado durante este emparejamiento.

5. Pulsar `Conectar` en la tarjeta de la cámara. El *chip* BLE de la cabecera pasará a verde.
6. En la sección `Cámara` que aparece, pulsar `WiFi AP` para activar el punto de acceso.
7. Pulsar `Leer credenciales WiFi`. Aparecerán SSID y contraseña.
8. En la sección `WiFi`, pulsar `Conectar al WiFi de la GoPro`. Android mostrará un diálogo del sistema solicitando confirmación; aceptar. El *chip* WiFi pasará a verde.
9. En la sección `Stream`, pulsar `Iniciar`. Tras unos segundos, el vídeo aparecerá en el área superior con las detecciones superpuestas si el modelo identifica algún objeto de las clases entrenadas.
10. Para detener, pulsar `Detener`. Para desconectar, pulsar `Desconectar` en la sección `Conexión`.

B.6. Reentrenamiento del modelo

En caso de querer reentrenar el modelo con un conjunto de datos ampliado o modificado, los pasos son los siguientes.

B.6.1. Descarga y preparación del *dataset*

1. Configurar AWS CLI para acceso anónimo al *bucket* público de AOT:
   ```
   aws s3 ls s3://airborne-obj-detection-challenge-training/ \
       --no-sign-request
   ```
2. Descargar las secuencias deseadas con `aws s3 cp --recursive`, especificando los identificadores de vuelo en el `groundtruth.csv` maestro.
3. Ejecutar el *notebook* de preparación `aot_to_yolo.ipynb` localmente (Jupyter). El *notebook* realiza el filtrado, la conversión a formato YOLO y la generación del *split train/val*.
4. Comprimir el directorio resultante `dataset-yolo-flat/` a un archivo ZIP.
5. Subir el ZIP a Google Drive en la carpeta `TFM-Airprox/`.

B.6.2. Entrenamiento en Google Colab

1. Abrir el *notebook* `airprox_yolov8_train.ipynb` en Google Colab.

2. Cambiar el *runtime* a GPU (Entorno de ejecución → Cambiar tipo de entorno → L4 o A100 si están disponibles).
3. Ejecutar las celdas en orden: montar Drive, descomprimir *dataset*, reescribir el *yaml* con las rutas de Colab, instalar Ultralytics y lanzar el entrenamiento.
4. El *callback* configurado copia los pesos (`best.pt` y `last.pt`) a Drive tras cada época para prevenir pérdidas por desconexión.
5. Al terminar, ejecutar la celda de exportación a LiteRT (`model.export(format='tflite', imgsz=640)`).

B.6.3. Integración del nuevo modelo en la aplicación

1. Descargar el archivo `best.tflite` generado y renombrarlo si se desea.
2. Sustituir el archivo actual en la ruta `app/src/main/assets/best.tflite` del proyecto Android.
3. Recompilar e instalar la aplicación. El nuevo modelo se cargará automáticamente al arrancar.

En caso de haber modificado el número de clases o los nombres, ajustar las constantes correspondientes en `GoProDetector.kt` (`NUM_CLASSES` y `CLASS_LABELS`).

B.7. Consideraciones de compatibilidad hardware

Durante el desarrollo del proyecto se detectó una limitación específica en Xiaomi Redmi Note 8 con MediaTek Helio G85 sobre HyperOS, en la que la captura de fotogramas del *stream* de LibVLC resulta intermitente. La aplicación funciona pero las detecciones son esporádicas. Se recomienda utilizar dispositivos con *System-on-Chip* Snapdragon (validado sobre Snapdragon 678 en Xiaomi Redmi Note 10) para operación estable. La resolución completa de esta limitación pasa por la interceptación directa de *buffers* de vídeo de LibVLC, línea de trabajo futura documentada en el Capítulo 7.

C. Estimación de costes

Este anexo recoge una estimación de los costes asociados al presente TFM desde tres perspectivas complementarias: el coste de despliegue del sistema propuesto (*hardware* necesario para el usuario final), una comparación de ese coste frente a la alternativa Aerial Guard, y un análisis del coste de reentrenamiento del modelo con mayor volumen de datos y recursos de cómputo. El objetivo es dar una idea del coste del sistema y servir de referencia para futuros desarrollos, sin entrar en un análisis económico detallado.

C.1. Coste de despliegue del sistema

La solución propuesta requiere dos elementos *hardware*: una cámara GoPro Hero 10 o superior (compatible con la interfaz Open GoPro) y un teléfono Android con soporte de LiteRT y aceleración mediante XNNPACK. La Tabla C.1 resume los precios de referencia a fecha de redacción de este TFM.

Elemento	**Config. gama media**	**Config. gama alta**
Cámara GoPro Hero 12 Black	450 €	450 €
Teléfono Android (gama media) p. ej. Google Pixel 7a	400 €	–
Teléfono Android (gama alta) p. ej. Samsung Galaxy S24	–	900 €
Total	**850 €**	**1 350 €**

Tabla C.1: Coste estimado del *hardware* necesario para desplegar el sistema propuesto, con una configuración de gama media y otra de gama alta.

Como es posible que parte de los pilotos ya cuente con un móvil adecuado, el coste

añadido se limitaría al de la cámara deportiva, que en algunos casos tampoco supondría un gasto nuevo.

C.2. Comparación con Aerial Guard

La alternativa más próxima en el mercado es Aerial Guard [4, 5], aplicación disponible exclusivamente para iPhone y que utiliza la cámara del propio teléfono. La Tabla C.2 resume la comparación.

	Sistema propuesto	**Aerial Guard**
Plataforma	Android	iOS (solo iPhone)
Cámara	Externa (GoPro)	Integrada del teléfono
Coste de *hardware*	850–1 350 €	900–1 500 €(iPhone)
Posicionamiento cámara	Flexible	Limitado por soporte
Teléfono libre en vuelo	Sí	No (ocupado por la cámara)
Métricas públicas	$mAP@0.5 = 0.993$ (clase `aircraft`)	No disponibles

Tabla C.2: Comparación de coste y características principales entre la solución propuesta y Aerial Guard.

En términos de coste bruto ambas soluciones son comparables si el usuario parte sin dispositivo. La ventaja diferencial del sistema propuesto no es económica, sino funcional: la cámara externa permite escoger el punto de montaje óptimo en la aeronave sin inmovilizar el teléfono, que queda libre para navegación, radio o cartografía durante el vuelo. En cuanto a capacidad de detección, Aerial Guard no publica métricas verificables, por lo que una comparación cuantitativa objetiva queda fuera del alcance de este TFM.

C.3. Coste de reentrenamiento del modelo

En este TFM el modelo se ha entrenado con un subconjunto de 4 784 imágenes del *dataset* AOT, utilizando Google Colab Pro con una GPU NVIDIA L4. En esta configuración, un entrenamiento completo (25 épocas, 1280 px) requiere unas 2–3 horas de GPU, con un coste directo marginal gracias a la suscripción mensual de Colab Pro (12 €/mes). La Tabla C.3 estima el coste de escenarios de reentrenamiento progresivamente más ambiciosos, de cara a futuras evoluciones del sistema.

Escenario	Datos	Tiempo GPU	Coste estimado
Entrenamiento actual (TFM)	4 784 img.	2–3 h en L4	<5 €
Entrenamiento ampliado	∼50 000 img.	20–30 h en A100	60–120 €
Entrenamiento con subconjunto AOT	∼500 000 img.	∼5 días en A100	400–700 €
Entrenamiento con AOT completo	∼5–6 M img.	∼3–4 sem. en A100	2 500–4 500 €

Tabla C.3: Estimación aproximada del coste de reentrenamiento del modelo según el volumen de datos y los recursos de cómputo utilizados. Precios de referencia de GPU en la nube (vast.ai / RunPod / Google Cloud) a fecha de 2026.

Las cifras son aproximadas y dependen del modelo (YOLOv8n frente a variantes más grandes), de la resolución de entrenamiento (640 vs. 1280 px) y del proveedor de *cloud* elegido. El escenario más ambicioso, entrenar con la totalidad del *dataset* AOT (~5 millones de imágenes anotadas), queda fuera del alcance de un TFM, pero resulta accesible en el marco de un proyecto de investigación o de una inversión comercial.

C.4. Conclusiones del análisis de costes

De la estimación anterior se desprenden tres ideas de cara a la evolución del sistema:

- El coste de despliegue del sistema propuesto es equiparable al de la alternativa comercial de referencia, con ventajas funcionales relevantes gracias al uso de cámara externa.
- El coste de reentrenamiento con el subconjunto actual es despreciable, lo que facilita iteraciones rápidas sobre el modelo durante el desarrollo.
- Escalar a volúmenes mayores de datos exige una inversión moderada en cómputo *cloud*, pero sigue siendo asumible: el salto cualitativo que podría suponer entrenar con la totalidad del *dataset* AOT, estimado entre 2 500 y 4 500 €, lo convierte en el germen natural de un TFM posterior o de una futura línea de investigación o producto comercial.